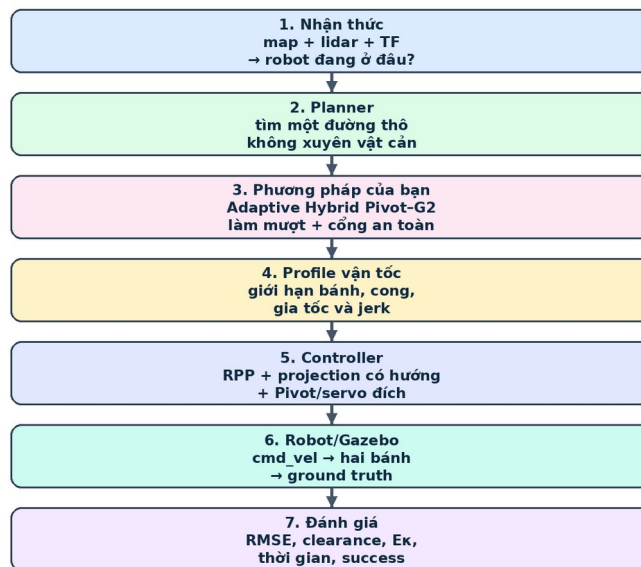


BÁO CÁO NGHIÊN CỨU KHOA HỌC

ADAPTIVE HYBRID PIVOT-G² CHO ROBOT HAI BÁNH VI SAI TRONG ROS 2 / NAV2

TỪ GOAL TRONG RVIZ2 ĐẾN CHUYỂN ĐỘNG THẬT TRONG GAZEBO



Mỗi khối đều được giải thích từ khái niệm cơ bản đến công thức và dữ liệu đo.

Luồng tổng quát của dự án, từ goal trong RViz2 đến chuyển động vật lý và dữ liệu đánh giá trong Gazebo.

Tài liệu này được viết theo đúng mạch phát triển của dự án: trước tiên hiểu Nav2 đang làm gì, sau đó xác định chính xác đầu vào và đầu ra của từng khâu, rồi mới giải thích vì sao xuất hiện ý tưởng Pivot-G², cách thuật toán được xây dựng, cách nó được tích hợp vào ROS 2 và cuối cùng là cách kiểm chứng bằng số liệu. Mục tiêu của bản báo cáo không phải chỉ để mô tả một thuật toán đã hoàn thành, mà còn để lưu lại toàn bộ quá trình suy luận để sau này nhìn lại vẫn hiểu tại sao từng thành phần có mặt trong hệ thống.

Bản đang đọc là bản rà soát toàn diện ngày 26/07/2026. Toàn bộ nội dung của bản gốc được giữ nguyên để không làm mất mạch suy luận đã hình thành trong quá trình nghiên cứu. Những phần được bổ sung tập trung vào bốn vấn đề đã bộc lộ khi quan sát đồng thời RViz2, Gazebo và dữ liệu trace: neo chính xác start-goal của path, xác định hướng xuất phát, phanh khi quay, và hợp đồng phần cứng của hai động cơ GA25 cùng bộ nguồn 4S4P. Kết quả cũ ngày 25/07 vẫn được giữ như dấu vết phát triển; kết quả hiện tại được ghi riêng để người đọc không nhầm hai phiên bản.

I, Đặt vấn đề

Trước khi nghĩ ra ý tưởng để làm thuật toán cho một AMR áp dụng ROS 2 thì phải tìm hiểu thật kĩ về kiến trúc của hệ Nav2, vì đây là hệ thống trực tiếp quyết định robot sẽ nhận mục tiêu như thế nào, tìm đường ra sao, bám đường bằng cách nào và xử lý khi gặp tình huống bất thường như thế nào. Nếu chưa hiểu kiến trúc này mà đã bắt đầu sửa thuật toán thì rất dễ rơi vào tình trạng thay đổi một tham số nhưng không biết nó đang tác động vào planner, smoother hay controller.

Nav2 có rất nhiều nội dung phải đọc, từ Behavior Tree, lifecycle node, TF, costmap (lưới chi phí biểu diễn vật cản và mức độ nguy hiểm) cho đến các plugin lập đường và điều khiển. Trong số đó, navigation plugins là phần quan trọng nhất đối với đề tài này, bởi Nav2 không ép người dùng phải sử dụng một thuật toán duy nhất. Planner, smoother, controller, goal checker và behavior đều được tổ chức thành các plugin tương đối độc lập. Nhờ đó mình có thể giữ nguyên những thành phần đã ổn định và chỉ thay thế đúng phần muốn nghiên cứu.

Trong cách hiểu đơn giản nhất, global planner (bộ lập đường toàn cục) có nhiệm vụ tìm một đường hình học từ vị trí hiện tại đến vị trí đích trên global costmap. Smoother nhận đường đó và cố gắng giảm các góc gãy hoặc làm đường dễ thực hiện hơn. Controller chạy ở tần số cao hơn, liên tục quan sát pose của robot và phát ra lệnh vận tốc tuyến tính, vận tốc góc. Behavior được dùng khi robot cần quay, lùi, chờ hoặc thoát khỏi một trạng thái thất bại. Các khâu này liên kết với nhau nhưng không phải là một thuật toán duy nhất.

Nếu không hiểu rõ từng khâu thì mình chỉ đang chạy mò cấu hình mặc định. Cấu hình mặc định có thể tạo cảm giác robot chạy tốt trong một vài cảnh mô phỏng, nhưng khi thay bản đồ, thay kích thước robot, tăng vận tốc hoặc đưa lên robot thật thì các vấn đề bắt đầu lộ ra: robot cắt góc, quay quá nhiều, rung lắc, bám sai nhánh, đi sát kè hoặc đến gần đích nhưng không ổn định được hướng.

Qua phân tích kiến trúc Nav2, có thể thấy hệ thống đã cung cấp tương đối đầy đủ các thành phần cho bài toán dẫn đường. Tuy nhiên, việc tìm được một đường không va chạm chưa đồng nghĩa với việc robot sẽ thực hiện đường đó tốt. Một đường đi có thể hợp lệ trên costmap nhưng vẫn chứa góc gấp, có độ cong thay đổi đột ngột, đi quá sát vật cản hoặc đòi hỏi controller phải tự sửa rất nhiều trong quá trình bám.

Đối với robot hai bánh vi sai, vấn đề thể hiện rõ nhất tại các góc cua. Robot loại này không thể dịch ngang tức thời. Muốn đổi hướng, nó phải tạo chênh lệch vận tốc giữa bánh trái và bánh phải. Khi đường tham chiếu đổi hướng đột ngột, controller buộc phải lựa chọn giữa giảm tốc mạnh, quay tại chỗ, cắt góc hoặc tạo ra một đường cong bám thực tế khác với đường planner đã trả về.

Nếu toàn bộ quyết định này được giao cho controller thì hành vi phụ thuộc rất nhiều vào lookahead (khoảng nhìn trước), giới hạn gia tốc, costmap cục bộ, tần số điều khiển và dự báo va chạm. Cùng một raw path (đường đi thô trước khi làm mượt) nhưng chỉ cần thay một vài tham số là robot có thể chuyển từ đi tương đối trơn sang dao động, dừng-quay liên tục hoặc báo

không thể điều khiển. Vì vậy khoảng trống nghiên cứu không nằm ở việc xây dựng lại toàn bộ global planner, mà nằm ở khâu biến đường hình học do planner tạo ra thành một đường và một quy luật vận tốc mà robot vi sai có thể thực hiện an toàn.

Vị trí can thiệp chính vì thế nằm ở Smoother Server (máy chủ chạy các plugin làm mượt) và phần giao tiếp giữa smoother với controller. Smoother chịu trách nhiệm thay đổi hình dạng của đường. Controller chịu trách nhiệm thực hiện đường đó bằng lệnh vận tốc. Nếu chỉ làm mượt hình học mà không xét profile vận tốc thì robot vẫn có thể vào cua quá nhanh. Ngược lại, nếu chỉ giảm tốc trong controller mà giữ nguyên mọi góc gấp thì robot sẽ an toàn hơn nhưng mất nhiều thời gian và chuyển động bị gián đoạn.

Từ đây bài toán được đặt ra như sau: với một raw path bất kỳ do global planner tạo ra, tại mỗi góc cua cần quyết định robot nên dừng để quay tại chỗ hay nên thay góc gấp bằng một đoạn cong chuyển tiếp. Quyết định đó không thể chỉ dựa vào độ lớn của góc, mà còn phải xét khoảng trống cục bộ, footprint (đa giác chiếu bằng của toàn thân robot) của robot, vận tốc bánh xe, độ cong của đường, khả năng phanh trước góc và khả năng controller thực hiện trong vòng kín.

Ý chính của đề tài không phải là làm cho đường nhìn đẹp hơn trên RViz2. Ý chính là biến một đường planner hợp lệ về mặt hình học thành một chuyển động thực sự có thể chạy được, có kiểm tra toàn thân robot, có giới hạn vận tốc và có phương án dự phòng khi nhánh làm mượt không phù hợp.

II, Từ một goal trong RViz2 đến raw path mà global planner trả về

2.1 Người dùng đặt goal thì Nav2 thực sự làm gì?

Khi người dùng dùng công cụ 2D Goal Pose trong RViz2 để chọn một vị trí và kéo chuột xác định hướng đích, dữ liệu được gửi vào hệ thống dưới dạng một pose trong frame map. Behavior Tree Navigator nhận nhiệm vụ và gọi Planner Server thông qua action ComputePathToPose hoặc một action tương đương. Planner Server lấy pose bắt đầu, pose đích và global costmap để chạy plugin planner đang được chọn.

Điểm quan trọng ở đây là global planner không trực tiếp phát vận tốc cho robot. Nó chỉ tạo ra một đường hình học. Sau khi đường được tạo, Behavior Tree mới chuyển kết quả sang Smoother Server nếu cần làm mượt, rồi chuyển tiếp sang Controller Server (máy chủ chạy bộ điều khiển bám đường) để thực hiện FollowPath. Vì vậy cần tách rõ ba khái niệm: tìm đường, làm đường để thực hiện hơn và điều khiển robot chạy theo đường.

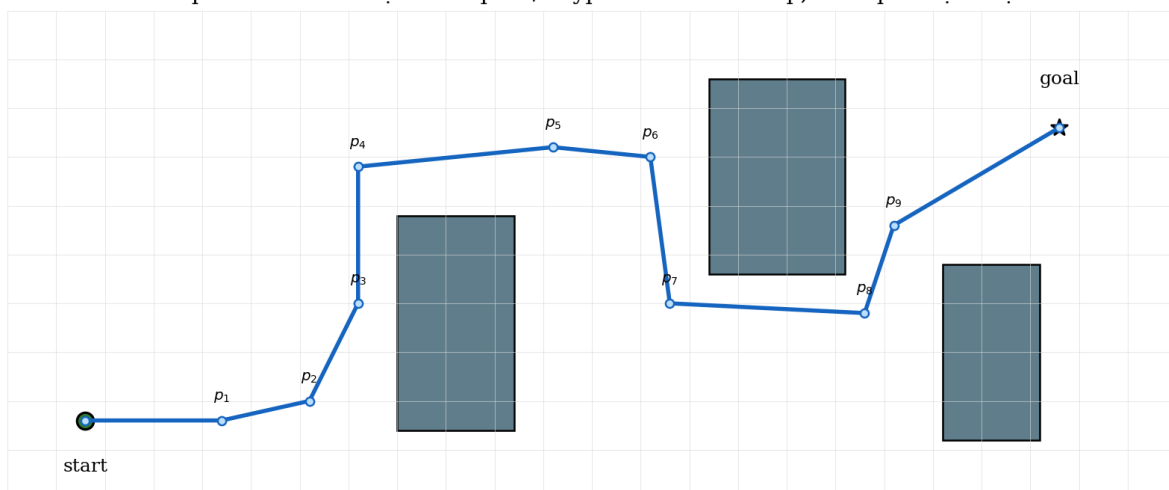
Trong Nav2, kết quả của global planner thường được trả về bằng thông điệp nav_msgs/Path. Thông điệp này chứa header và một mảng các geometry_msgs/PoseStamped. Mỗi PoseStamped có tọa độ vị trí và orientation. Đối với bài toán chuyển động phẳng, có thể hiểu mỗi phần tử là một pose p_i gồm x_i , y_i và góc hướng ψ_i .

$$P_{\text{raw}} = \{p_0, p_1, \dots, p_N\}, \quad p_i = (x_i, y_i, \psi_i)$$

Trong đó: P_{raw} là raw path gồm $N+1$ pose; p_i là pose thứ i ; x_i và y_i là tọa độ mét trong frame map; ψ_i là góc yaw tham chiếu tại pose i .

Ý nghĩa và lý do sử dụng: Công thức này mô tả đúng cấu trúc hình học mà global planner trả về. Nó chưa chứa thời gian, vận tốc hay gia tốc, vì vậy chưa phải trajectory hoàn chỉnh.

Global planner trả về một chuỗi pose/waypoint trên costmap, chưa phải lệnh vận tốc



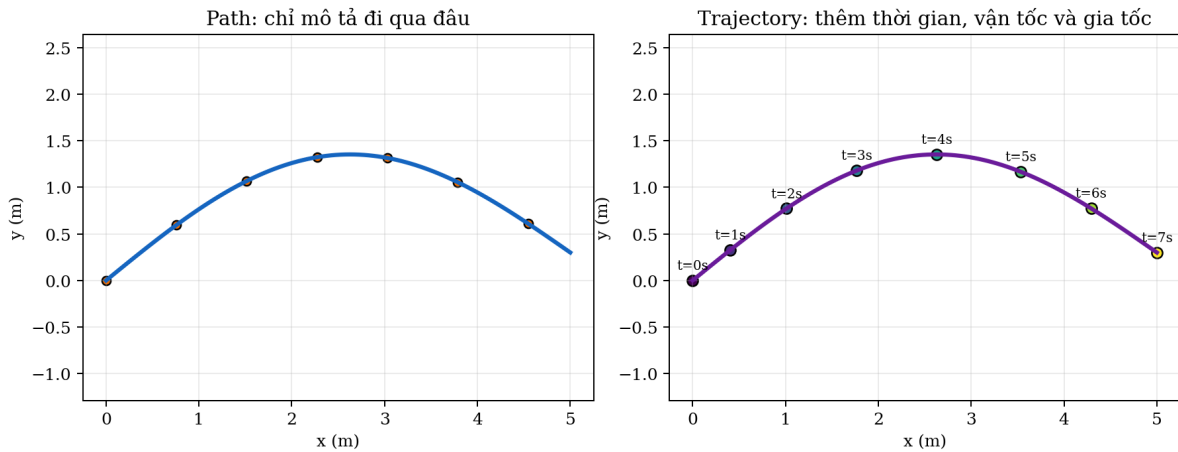
Minh họa một raw path gồm các waypoint mà global planner trả về trên costmap. Hình được tự vẽ lại theo cấu trúc nav_msgs/Path.

Các pose được sắp theo thứ tự từ start đến goal. Khi chỉ nhìn phần vị trí và nối hai pose liên tiếp bằng đoạn thẳng, ta nhận được một polyline (đường gấp khúc gồm các đoạn thẳng). Mỗi

điểm trong chuỗi thường được gọi là waypoint (điểm mẫu trên đường). Waypoint ở đây không nhất thiết là một mục tiêu độc lập mà robot phải dừng tại đó; nó chỉ là một mẫu hình học dùng để biểu diễn đường.

Raw path là đường được lấy trực tiếp từ planner trước mọi bước làm mượt. Việc gọi nó là “thô” không có nghĩa planner làm sai. Nó chỉ có nghĩa đường vẫn mang đặc điểm của biểu diễn lưới, độ phân giải costmap, cách nội suy của planner và các quy tắc tối ưu riêng của planner. Raw path là đầu vào chung để đánh giá các smoother một cách công bằng.

2.2 Path khác trajectory ở điểm nào?



Path chỉ mô tả hình học, còn trajectory gắn thêm thời gian và quy luật vận tốc. Hình tự dựng để phân biệt hai khái niệm.

Path chỉ trả lời câu hỏi robot cần đi qua đâu. Trajectory phải trả lời thêm robot đi đến từng vị trí vào thời điểm nào, với vận tốc tuyến tính bao nhiêu, vận tốc góc bao nhiêu và có thể cả gia tốc bao nhiêu. Một path hoàn toàn có thể không chứa thông tin thời gian. Nếu đưa thẳng path đó cho controller, controller phải tự suy ra lệnh vận tốc trong quá trình chạy.

Sự khác biệt này rất quan trọng đối với Pivot- G^2 . Phần hình học của thuật toán tạo ra một path đã được thay đổi tại các góc. Phần profile vận tốc mới biến path thành một quy luật chuyển động khả thi. Vì vậy không nên gọi riêng đường Bézier là trajectory (quỹ đạo có gắn thời gian và vận tốc) hoàn chỉnh nếu chưa có time-parameterization.

2.3 Vì sao đường planner trả về thường có nhiều điểm và góc gấp?

Global costmap là lưới rời rạc. Với planner 2D, trạng thái thường gắn với ô lưới và các chuyển động lân cận. Vì vậy, ngay cả khi đường thực tế có thể đi theo một đường thẳng, planner vẫn có thể trả về nhiều pose rất gần nhau. Một số đoạn có nhiều điểm gần thẳng hàng, một số đoạn thay đổi hướng trái-phải nhỏ do đường bám theo biên ô lưới, và tại chỗ chuyển hành lang có thể xuất hiện góc gấp rõ rệt.

Các planner khác nhau cho raw path khác nhau. NavFn tạo đường kiểu wavefront trên lưới. Theta* có thể nối tắt bằng line-of-sight nên thường cho đoạn thẳng dài hơn và góc ít mang hình bậc thang hơn. Smac Planner 2D là A* có xét costmap. Smac Hybrid và State Lattice đưa

hướng vào trạng thái, nhưng chúng cũng không bảo đảm raw path cuối cùng đã phù hợp hoàn toàn với controller và footprint trong mọi cấu hình.

Do đó smoother của dự án được thiết kế như một bộ hậu xử lý, không phụ thuộc cứng vào một planner. Cùng một thuật toán có thể nhận raw path từ NavFn A*, NavFn Dijkstra, Theta*, Smac 2D hoặc Smac Hybrid. Khi benchmark, planner ID và dấu vân tay raw_path_sha256 được lưu lại để chắc chắn các phương pháp được so sánh trên đúng cùng một đầu vào.

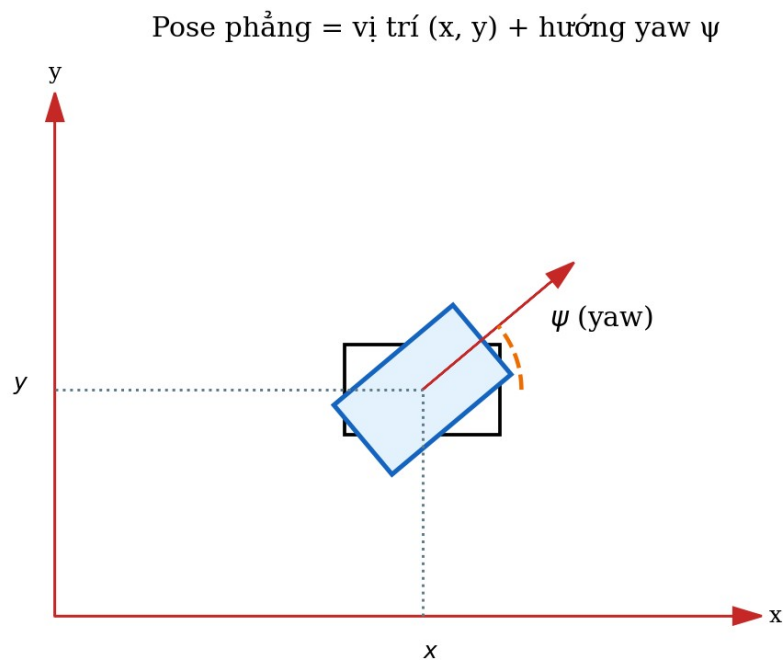
Planner	Biểu diễn chính	Ý nghĩa đối với dự án
NavFn A*/Dijkstra	Lưới 2D	Đường cổ điển, đơn giản, thường có nhiều waypoint và phụ thuộc inflation.
Theta*	Lưới 2D có line-of-sight	Đường any-angle, đoạn thẳng dài hơn nhưng vẫn chưa chứa quy luật vận tốc.
Smac Planner 2D	A* có xét costmap	Đường 2D có xu hướng tránh vùng cost cao, phù hợp robot tròn hoặc vi sai.
Smac Hybrid-A*	Không gian SE(2)	Có hướng và footprint-aware, nhưng phức tạp hơn và không phải lúc nào cũng phù hợp nhất với robot vi sai quay tại chỗ.
Smac State Lattice	SE(2) với motion primitive	Linh hoạt theo bộ primitive nhưng phụ thuộc mạnh vào file lattice và độ phân giải.

III, Robot hai bánh vi sai thực sự chuyển động như thế nào?

3.1 Pose, yaw và quaternion

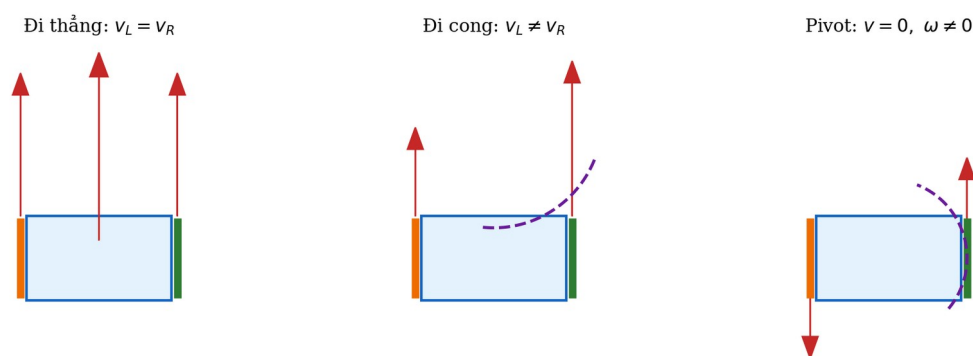
Trong bài toán phẳng, pose của robot có thể hiểu là bộ ba (x, y, ψ) , trong đó x và y là vị trí của tâm robot còn ψ là yaw, tức góc hướng của thân xe quanh trục z . ROS vẫn lưu orientation bằng quaternion để dùng chung cho không gian 3D. Khi robot không nghiêng theo roll và pitch, quaternion của yaw có thể biểu diễn bằng $q_z = \sin(\psi/2)$ và $q_w = \cos(\psi/2)$.

Khi tính sai số góc, không thể trừ hai góc một cách ngây thơ vì góc nằm trên vòng tròn. Ví dụ $+179^\circ$ và -179° thực chất chỉ lệch 2° , không phải 358° . Vì vậy mọi sai số yaw trong hệ thống đều cần chuẩn hóa bằng $\text{atan2}(\sin \Delta\psi, \cos \Delta\psi)$ để đưa về khoảng $-\pi$ đến π .



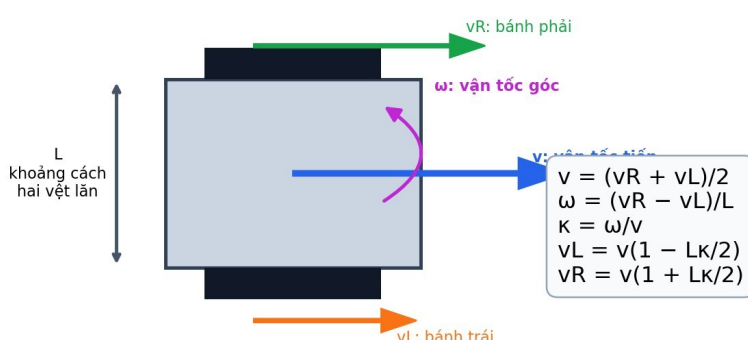
Pose phẳng được mô tả bởi tọa độ x, y và góc yaw ψ trong một frame tọa độ.

3.2 Phương trình động học cơ bản



Ba trạng thái cơ bản của robot vi sai: đi thẳng, đi cong và Pivot quay tại chỗ.

Robot vi sai: không trượt ngang lý tưởng; muốn quay phải tạo chênh lệch tốc độ hai bánh.



Mô hình động học của robot vi sai và mối liên hệ giữa vận tốc thân xe với vận tốc hai bánh trong báo cáo dự án.

Gọi v_R và v_L là vận tốc dài tại vành bánh phải và bánh trái, L là khoảng cách giữa hai mặt phẳng tâm vệt lăn. Vận tốc tuyến tính của tâm robot là trung bình của hai vận tốc bánh, còn vận tốc góc xuất hiện do sự chênh lệch giữa chúng.

$$v = (v_R + v_L) / 2$$

Trong đó: v là vận tốc tuyến tính của tâm robot; v_R và v_L là vận tốc dài tại vành bánh phải và bánh trái.

Ý nghĩa và lý do sử dụng: Tâm robot tiến theo vận tốc trung bình của hai bánh. Đây là phương trình cơ bản nhất để chuyển từ không gian vận tốc bánh sang vận tốc thân xe.

$$\omega = (v_R - v_L) / L$$

Trong đó: ω là vận tốc góc quanh trục z; L là khoảng cách giữa hai vệt lăn; v_R-v_L là độ chênh vận tốc hai bánh.

Ý nghĩa và lý do sử dụng: Robot vi sai đối hướng nhờ chênh lệch vận tốc bánh. L càng lớn thì cùng một độ chênh bánh tạo ra vận tốc góc nhỏ hơn.

$$\dot{x} = v \cos \psi, \quad \dot{y} = v \sin \psi, \quad \dot{\psi} = \omega$$

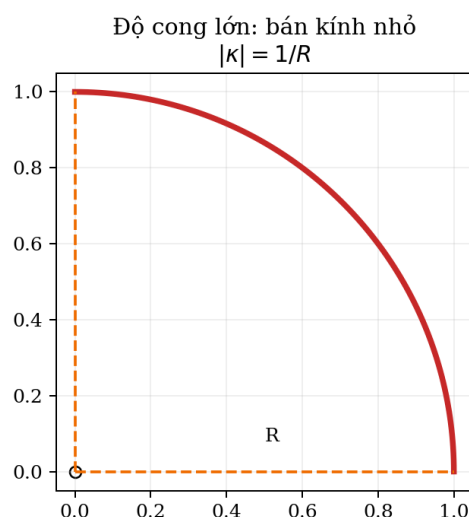
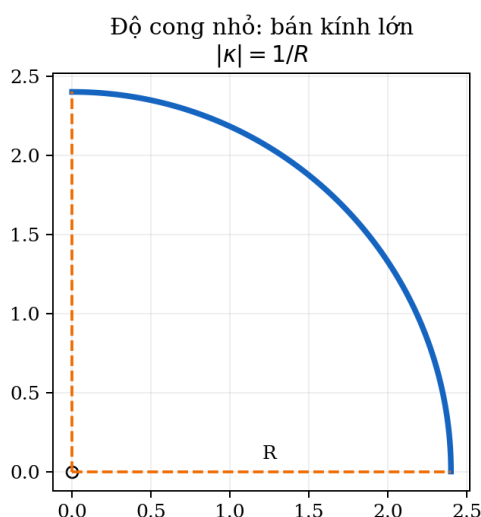
Trong đó: $\dot{x}$ và $\dot{y}$ là tốc độ thay đổi tọa độ x và y; $\dot{\psi}$ là tốc độ thay đổi yaw; v là vận tốc tuyến tính; ψ là hướng hiện tại; ω là vận tốc góc.

Ý nghĩa và lý do sử dụng: Ba phương trình biến lệnh v, ω thành chuyển động trên mặt phẳng. Chúng cũng cho thấy robot không có thành phần vận tốc ngang độc lập.

Nếu hai bánh chạy cùng tốc độ và cùng chiều thì ω bằng 0, robot đi thẳng. Nếu một bánh nhanh hơn bánh còn lại thì robot đi theo đường cong. Nếu hai bánh quay ngược chiều với độ lớn bằng nhau thì v bằng 0 nhưng ω khác 0, tâm robot gần như đứng yên và thân xe quay tại chỗ. Trạng thái cuối cùng chính là Pivot.

Việc robot có thể quay tại chỗ là một lợi thế rất lớn trong hành lang hẹp. Tuy nhiên, quay tại chỗ ở mọi góc lại làm chuyển động bị ngắt thành nhiều pha đi–dừng–quay–đi. Vì vậy Pivot không phải lúc nào cũng là lựa chọn tốt nhất; nó cần được giữ như một trạng thái khả thi để sử dụng đúng lúc.

3.3 Độ cong và giới hạn vận tốc bánh



Độ cong là nghịch đảo bán kính đối với cung tròn: bán kính nhỏ tương ứng với góc cua gắt hơn.

Khi robot đang chuyển động với v khác 0, độ cong κ của quỹ đạo liên hệ với vận tốc góc theo $\kappa = \omega/v$. Độ cong càng lớn thì robot đổi hướng càng nhanh trên mỗi mét đường đi. Từ phương trình động học có thể suy ra vận tốc hai bánh khi robot bám một đường có độ cong κ .

$$\kappa = \omega / v, \quad \omega = v\kappa$$

Trong đó: κ là độ cong với đơn vị 1/m; ω là vận tốc góc rad/s; v là vận tốc tuyến tính m/s và phải khác 0.

Ý nghĩa và lý do sử dụng: Độ cong nối trực tiếp hình học đường với lệnh điều khiển. Nếu κ thay đổi đột ngột khi v còn lớn thì ω yêu cầu cũng thay đổi đột ngột.

$$v_L = v(1 - L\kappa/2), \quad v_R = v(1 + L\kappa/2)$$

Trong đó: v_L, v_R là vận tốc bánh trái và phải; L là khoảng cách vệt lăn; κ là độ cong của đường; v là vận tốc tâm robot.

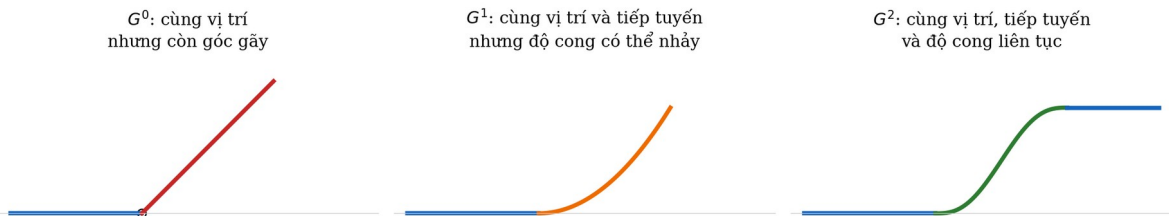
Ý nghĩa và lý do sử dụng: Công thức dùng để kiểm tra một đường cong có vượt vận tốc bánh hoặc buộc bánh trong đảo chiều hay không.

Hai phương trình này cho thấy một đường cong hình học không thể tách rời khỏi giới hạn cơ cấu chấp hành. Nếu $|L\kappa/2|$ lớn hơn 1 thì bánh phía trong phải đảo chiều. Trong lõi Pivot- G^2 , một transition (đoạn chuyển tiếp) được định nghĩa là chuyển động tiến liên tục sẽ bị loại nếu nó buộc bánh trong đảo chiều. Trường hợp cần đảo chiều được chuyển sang trạng thái Pivot thay vì cố coi đó là một đường cong tiến bình thường.

Với độ cong không đổi, bán kính quỹ đạo tâm bằng $1/|\kappa|$. Tuy nhiên tham số R trong thuật toán không phải bán kính của một cung tròn đầu ra. R chỉ là bán kính thiết kế dùng để suy ra khoảng trim d . Đường đầu ra thực sự là Bézier bậc năm với κ thay đổi dọc theo đường.

IV, Vì sao phải giải thích tính liên tục G^2 trước khi nói đến đường cong của phương pháp?

4.1 Không phải đường cong nào nhìn trơn cũng phù hợp với robot



So sánh trực quan liên tục G^0 , G^1 và G^2 . Dự án cần G^2 để độ cong không nhảy tại điểm nối với đoạn thẳng.

Nếu chỉ nhìn bằng mắt, một đường cong có thể có vẻ mềm hơn một góc vuông. Tuy nhiên controller không nhìn đường theo cảm giác. Nó cần hướng tiếp tuyến và độ cong để suy ra vận tốc góc. Nếu tại điểm nối giữa đoạn thẳng và đoạn cong, hướng hoặc độ cong nhảy đột ngột thì lệnh ω cũng có thể thay đổi đột ngột. Khi đó robot dễ phát sinh sai số bám, dao động hoặc phải giảm tốc mạnh.

Vì vậy cần phân biệt các mức liên tục hình học. G^0 chỉ yêu cầu hai đoạn gặp nhau tại cùng một vị trí. G^1 yêu cầu thêm hướng tiếp tuyến giống nhau nên không còn góc gãy. G^2 yêu cầu thêm độ cong bằng nhau tại điểm nối. Đối với một đoạn thẳng, độ cong bằng 0, nên một transition nối G^2 với đoạn thẳng phải bắt đầu và kết thúc với κ bằng 0.

$$\kappa(0) = 0, \quad \kappa(1) = 0$$

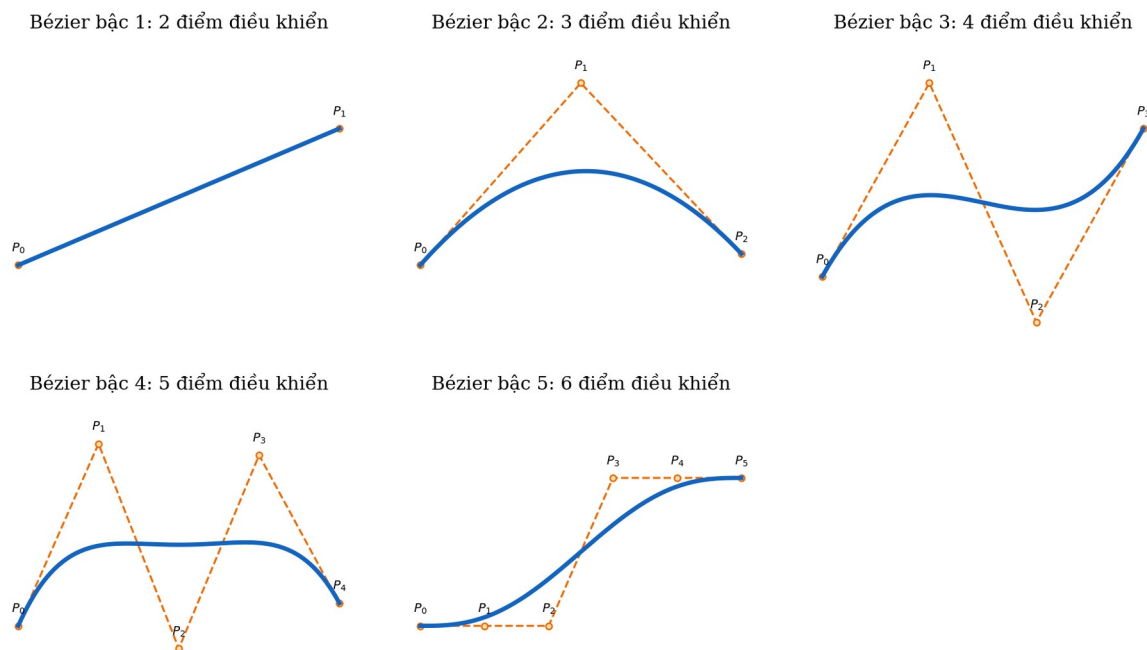
Trong đó: $\kappa(0)$ là độ cong tại đầu vào transition; $\kappa(1)$ là độ cong tại đầu ra transition.

Ý nghĩa và lý do sử dụng: Hai đoạn kề là đường thẳng nên có độ cong bằng 0. Muốn nối G^2 , transition cũng phải có độ cong bằng 0 ở hai đầu.

Ý nghĩa vật lý của điều kiện này là robot không bị yêu cầu chuyển tức thời từ đi thẳng sang một độ cong khác 0. Độ cong tăng dần khi vào cua và giảm dần khi ra khỏi cua. Đây là cơ sở để profile vận tốc và controller có thể tạo lệnh mượt hơn.

4.2 Từ Bézier bậc một đến Bézier bậc năm và lý do chọn bậc năm

Tăng bậc làm tăng số bậc tự do; bậc năm có 6 điểm để khống chế hai đầu



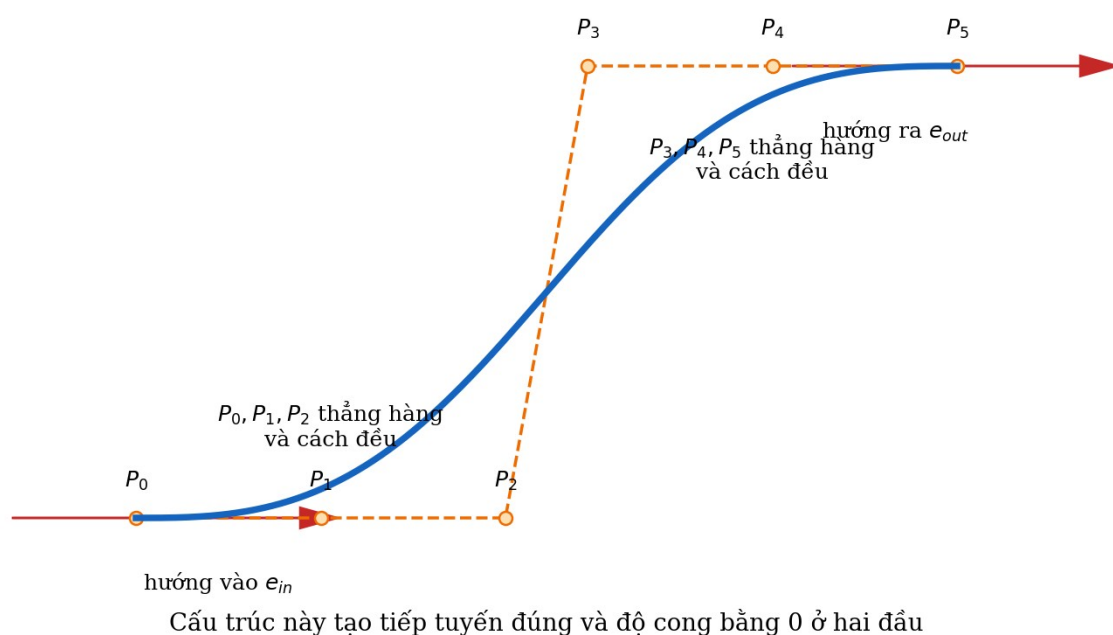
Bézier từ bậc một đến bậc năm. Bậc n có $n+1$ điểm điều khiển; bậc càng cao có nhiều bậc tự do hơn nhưng cũng khó kiểm soát hơn.

Đường Bézier bậc năm được xác định bởi sáu điểm điều khiển P_0 đến P_5 . Bậc năm đủ tự do để điều khiển vị trí, hướng và đạo hàm bậc hai ở hai đầu. Trong thiết kế của dự án, P_0 , P_1 , P_2 được đặt thẳng hàng theo hướng đi vào và cách đều nhau; P_3 , P_4 , P_5 được đặt thẳng hàng theo hướng đi ra và cũng cách đều. Cấu trúc này làm độ cong tại hai đầu bằng 0 và tạo nối G^2 với hai đoạn thẳng.

Bậc Bézier	Số điểm điều khiển	Điều khiển được ở hai đầu	Đánh giá cho bài toán
Bậc 1	2	Chỉ vị trí đầu và cuối	Chỉ là đoạn thẳng, không tạo được góc cong.
Bậc 2	3	Vị trí và một mức kéo hình dạng	Quá ít bậc tự do để đồng thời khống chế hai tiếp tuyến và độ cong.
Bậc 3	4	Vị trí và tiếp tuyến hai đầu	Phù hợp G^1 ; muốn ép độ cong hai đầu thường thiếu tự do hoặc cần ghép nhiều đoạn.
Bậc 4	5	Nhiều tự do hơn bậc 3	Có thể xây một số họ G^2 nhưng thiết kế endpoint không đối xứng và khó giải thích hơn.
Bậc 5	6	Vị trí, tiếp tuyến và đạo hàm bậc hai hai đầu	Phù hợp nhất với cấu trúc P_0, P_1, P_2 và P_3, P_4, P_5 thẳng

Bậc Bézier	Số điểm điều khiển	Điều khiển được ở hai đầu	Đánh giá cho bài toán
			hàng cách đều của dự án.

Lý do chọn bậc năm không phải vì bậc càng cao thì đường càng đẹp. Nếu tăng bậc quá mức, số điểm điều khiển và khả năng dao động cũng tăng, việc tìm kiếm trở nên khó hơn. Bậc năm là mức vừa đủ cho bài toán này: sáu điểm điều khiển cho phép dùng ba điểm đầu để khống chế điều kiện vào và ba điểm cuối để khống chế điều kiện ra. Nhờ cách đặt thẳng hàng và cách đều, đạo hàm bậc hai theo phương pháp tuyến tại hai đầu bị triệt tiêu, dẫn tới độ cong đều và cuối bằng 0.



Cấu trúc sáu điểm điều khiển của Bézier bậc năm được chọn để bảo đảm hướng và độ cong phù hợp ở hai đầu.

$$B(u) = \sum_{i=0}^5 C(5,i)(1-u)^{5-i}u^i P_i, \quad 0 \leq u \leq 1$$

Trong đó: $B(u)$ là vị trí trên đường Bézier; u là tham số chạy từ 0 đến 1; P_i là sáu điểm điều khiển; $C(5,i)$ là hệ số nhị thức $[1,5,10,10,5,1]$.

Ý nghĩa và lý do sử dụng: Bézier bậc năm có đủ sáu điểm điều khiển để khống chế vị trí, tiếp tuyến và đạo hàm bậc hai ở hai đầu, phù hợp mục tiêu nối G^2 với hai đoạn thẳng.

$$q = 0,35d$$

Trong đó: q là khoảng cách giữa các control point ở mỗi đầu; d là khoảng trim từ đỉnh góc đến entry/exit; 0,35 là hệ số thiết kế đang dùng.

Ý nghĩa và lý do sử dụng: q quyết định mức kéo của control polygon. Dự án dùng tỉ lệ cố định để giảm số biến tìm kiếm; biến thích nghi chính vẫn là d .

$$P0 = \text{entry}; \quad P1 = P0 + q \, e_{in}; \quad P2 = P0 + 2q \, e_{in}$$

Trong đó: $P0, P1, P2$ là ba điểm điều khiển đầu; entry là điểm bắt đầu transition; e_{in} là vector đơn vị hướng vào; q là khoảng cách control point.

Ý nghĩa và lý do sử dụng: Ba điểm thẳng hàng và cách đều theo hướng vào giúp tiếp tuyến đúng và triệt thành phần cong ở đầu transition.

$$P5 = \text{exit}; \quad P4 = P5 - q \, e_{out}; \quad P3 = P5 - 2q \, e_{out}$$

Trong đó: $P3, P4, P5$ là ba điểm điều khiển cuối; exit là điểm kết thúc transition; e_{out} là vector đơn vị hướng ra.

Ý nghĩa và lý do sử dụng: Cấu trúc đối xứng ở đầu ra giúp đường rời transition theo đúng hướng của đoạn thẳng sau và có độ cong về 0.

Trong đó d là khoảng cắt từ đỉnh góc về hai phía, q là khoảng cách giữa các control point đầu và cuối, e_{in} và e_{out} là hai vector đơn vị của hướng vào và hướng ra. Góc đối hướng φ được tính bằng atan2 của tích có hướng và tích vô hướng giữa hai vector này.

$$\varphi = \text{atan2}(\text{cross}(e_{in}, e_{out}), \text{dot}(e_{in}, e_{out}))$$

Trong đó: φ là góc đối hướng có dấu; cross cho biết chiều rẽ trái/phải; dot cho biết độ lớn góc; e_{in}, e_{out} là hai vector đơn vị.

Ý nghĩa và lý do sử dụng: Dùng atan2 thay vì arccos để giữ được dấu rẽ và tránh mất thông tin trái/phải.

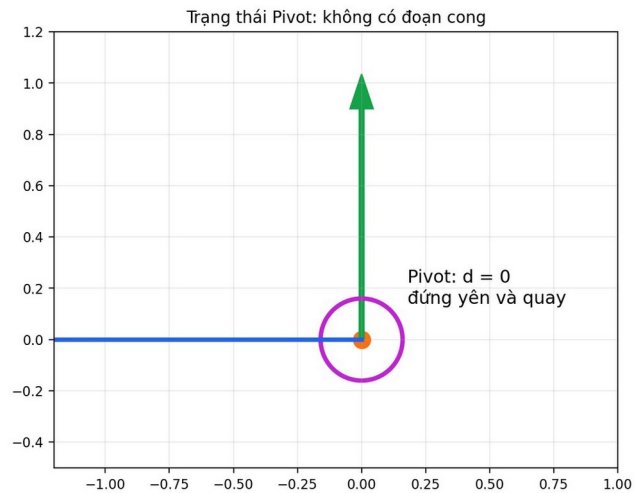
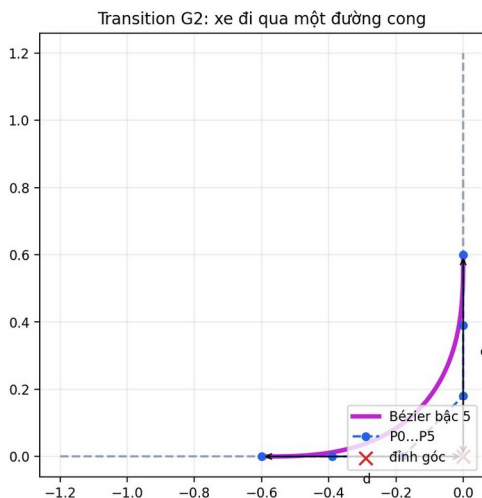
$$d = R \tan(|\varphi|/2) \quad \Leftrightarrow \quad R = d / \tan(|\varphi|/2)$$

Trong đó: d là khoảng trim; R là bán kính thiết kế; $|\varphi|$ là độ lớn góc đối hướng.

Ý nghĩa và lý do sử dụng: Quan hệ này xuất phát từ hình học tiếp tuyến của một góc. Thuật toán tối ưu d vì ràng buộc giữa hai góc kề trở thành phép cộng tuyến tính.

Tối ưu trực tiếp theo d thuận lợi hơn tối ưu theo R vì xung đột giữa hai góc kề nhau có thể viết thành một bất đẳng thức tuyến tính theo tổng hai khoảng trim. Adaptive mode vì thế tìm d riêng cho từng góc, rồi chỉ quy đổi về R để giải thích hình học.

Hai cách xử lý một góc của phương pháp Pivot-G2



Hai cách xử lý một góc: chuyển động theo transition Bézier G^2 hoặc giữ nguyên vị trí và thực hiện Pivot.

4.3 Độ cong và năng lượng độ cong

Độ cong của đường tham số $B(u)$ được tính từ đạo hàm bậc nhất và bậc hai. Giá trị này cho biết mức độ thay đổi hướng dọc theo đường. Để so sánh các ứng viên, dự án sử dụng năng lượng độ cong E_κ , là tích phân của κ^2 theo chiều dài cung.

$$\kappa(u) = (x'y'' - y'x'') / (x'^2 + y'^2)^{3/2}$$

Trong đó: x', y' là đạo hàm bậc nhất theo u ; x'', y'' là đạo hàm bậc hai; $\kappa(u)$ là độ cong có dấu.

Ý nghĩa và lý do sử dụng: Công thức đo mức đổi hướng của tiếp tuyến tại từng mẫu Bézier và được dùng để kiểm tra giới hạn bánh, tốc độ góc và gia tốc ngang.

$$E_\kappa = \int \kappa(s)^2 ds \approx \sum 0,5(\kappa_{i-1}^2 + \kappa_i^2)\Delta s$$

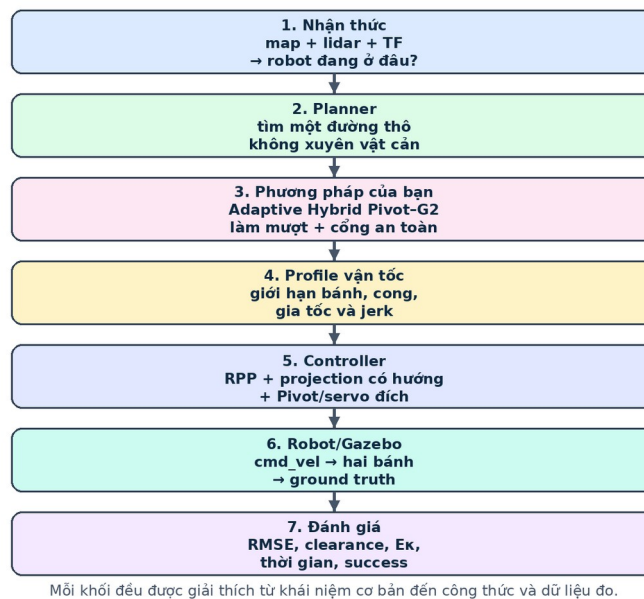
Trong đó: E_κ là năng lượng độ cong; s là chiều dài cung; Δs là khoảng cách giữa hai mẫu; κ_i là độ cong tại mẫu i .

Ý nghĩa và lý do sử dụng: Bình phương κ làm cả cong trái và cong phải đều bị phạt. E_κ nhỏ thường tạo đường ít gắt hơn nhưng không đồng nghĩa thời gian ngắn hoặc chắc chắn bám được.

E_κ nhỏ thường tương ứng với đường ít gắt hơn, nhưng E_κ không phải thời gian, không phải jerk (tốc độ thay đổi của gia tốc) và cũng không tự bảo đảm robot chạy được. Vì vậy nó chỉ là một thành phần trong objective hình học. Mọi ứng viên vẫn phải qua kiểm tra động học, time-parameterization và swept footprint (vùng toàn thân robot quét qua khi chuyển động).

V, Toàn bộ ý tưởng được chia thành ba lớp như thế nào?

TỪ GOAL TRONG RVIZ2 ĐẾN CHUYỂN ĐỘNG THẬT TRONG GAZEBO



Chuỗi xử lý nhìn từ goal trong RViz2 đến chuyển động vật lý và các đại lượng đánh giá.

5.1 Lớp hình học

Lớp hình học nhận nav_msgs/Path từ global planner. Raw path được làm sạch để loại pose trùng, điểm gần thẳng hàng và dao động trái-phải nhỏ. Sau đó các góc thực sự được phát hiện. Ở mỗi góc, thuật toán giữ một trạng thái Pivot và tạo nhiều ứng viên Bézier G^2 với các khoảng trim khác nhau. Mỗi ứng viên được kiểm tra bằng giới hạn động học, costmap và toàn bộ footprint. Những ứng viên an toàn được đưa vào quy hoạch động để chọn một chuỗi trạng thái cho toàn đường mà không làm hai đoạn cong chồng lên nhau.

Lớp hình học không chỉ tìm đường có E_k thấp. Sau khi tạo nhánh Pivot- G^2 , hệ thống còn so sánh với nhánh Nav2 Simple và giữ Raw làm phương án cuối. Đây là lý do phương pháp hoàn chỉnh có từ Hybrid: nó chủ động fallback (phương án dự phòng) thay vì ép mọi trường hợp phải dùng Pivot- G^2 .

5.2 Lớp động học và biên dạng vận tốc

Sau khi có path hình học, hệ thống tính trần vận tốc tại từng mẫu dựa trên v_{max} , ω_{max} , vận tốc bánh tối đa, gia tốc ngang và độ cong. Sau đó trần ở tương lai được truyền ngược để robot phanh trước góc hoặc trước goal; trần ở quá khứ được truyền xuôi để robot không tăng tốc quá nhanh ngay sau góc. Các lượt truyền được lặp đến khi hội tụ và tiếp tục bị giới hạn bởi gia tốc góc cùng jerk.

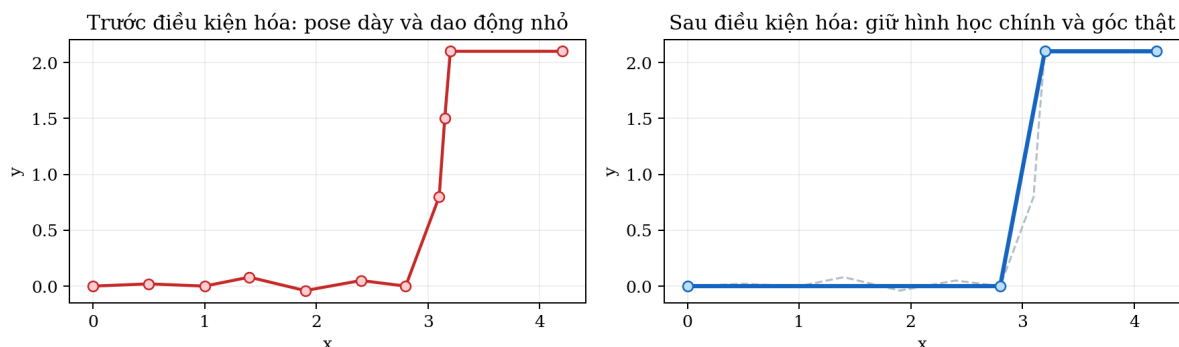
Nhờ vậy 0,30 m/s chỉ là vận tốc tối đa, không phải vận tốc cố định. Robot có thể chạy chậm hơn tại đường cong, gần vật cản, khi sai số bám lớn hoặc khi cần dừng để Pivot.

5.3 Lớp điều khiển vòng kín

Controller sử dụng RPP làm nền tảng nhưng được bổc thêm các cơ chế nhận biết maneuver. Phép projection (phép chiếu robot lên đường tham chiếu) không chỉ tìm điểm gần nhất mà còn xét hướng để tránh nhảy sang nhánh khác khi hai đoạn nằm gần nhau. Vận tốc mục tiêu là giá trị nhỏ nhất giữa cap hình học, cap của RPP, cap theo sai số bám và cap an toàn. Khi gặp marker Pivot, controller dừng tịnh tiến và quay đến yaw yêu cầu. Khi gần đích, terminal servo (điều khiển phản hồi cục bộ tại đích) tách pha lấy vị trí và pha căn yaw để tránh tình trạng robot quay đúng hướng nhưng bị trôi khỏi goal.

VI, Lớp hình học được xây dựng chi tiết như thế nào?

6.1 Điều kiện hóa raw path



Điều kiện hóa raw path loại pose dư và dao động nhỏ nhưng vẫn giữ start, goal và các góc có ý nghĩa.

Raw path có thể chứa hàng trăm pose. Nếu coi mọi thay đổi hướng nhỏ là một góc, smoother sẽ tạo quá nhiều transition và làm phức tạp cả hình học lẫn controller. Vì vậy bước đầu tiên là `condition_polyline`.

Thuật toán thử thay một chuỗi điểm bằng một chord nối điểm đầu và điểm cuối. Shortcut chỉ được chấp nhận khi mọi điểm bị bỏ không lệch chord quá ngưỡng, thứ tự đường không đổi, start và goal được giữ, và toàn chord đi qua predicate swept-footprint. Đối với dao động lưới trái-phải, shortcut chỉ được bật khi span, deviation, góc và số lần đổi dấu thỏa các ngưỡng được đặt trước.

Trong benchmark hình học chính, line-of-sight pruning được tắt đồng đều để mọi smoother nhận đúng cùng raw path. Nếu chỉ bật LOS cho Pivot- G^2 thì lợi ích của việc rút gọn raw path sẽ bị trộn với lợi ích của bản thân thuật toán làm mượt, khiến kết luận không công bằng.

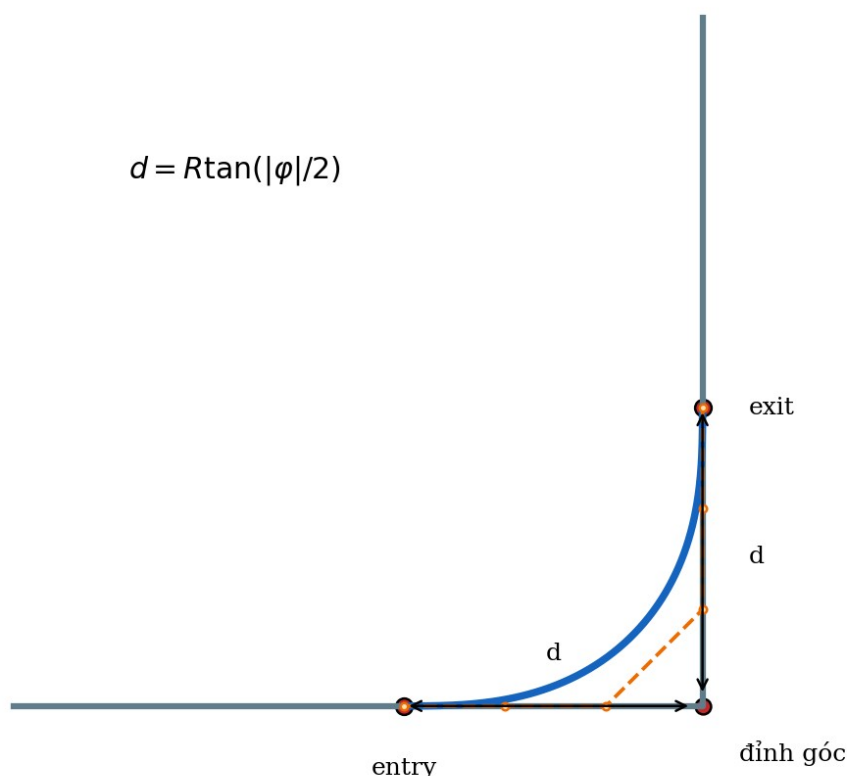
6.2 Phát hiện góc

Sau điều kiện hóa, với mỗi waypoint ở giữa, thuật toán lấy hướng của đoạn đi vào và đoạn đi ra. Góc đổi hướng φ được chuẩn hóa về khoảng $-\pi$ đến π . Dấu của φ cho biết rẽ trái hay rẽ phải; độ lớn $|\varphi|$ cho biết góc gắt đến mức nào.

Trong cấu hình hiện tại, góc nhỏ hơn 5° không cần tạo transition. Góc gần 180° nằm ngoài miền xử lý thích hợp của Bézier tiến và thường được giữ dưới dạng Pivot. Ngưỡng 5° không phải một định luật vật lý mà là ngưỡng chống việc thuật toán phản ứng với nhiễu hướng rất nhỏ của polyline.

6.3 Tạo trạng thái Pivot và miền ứng viên G^2

Khoảng trim d cắt góc gấp để đặt đoạn chuyển tiếp



Khoảng trim d cắt bớt hai đoạn thẳng quanh đỉnh để tạo entry và exit cho đoạn chuyển tiếp.

Mỗi góc luôn có một trạng thái Pivot với d bằng 0. Trạng thái này gần như không chiếm chiều dài của hai đoạn kề nên cũng là phương án giải xung đột khi hai góc nằm quá gần nhau.

Đối với nhánh G^2 , miền d được suy ra từ độ lớn góc, chiều dài hai đoạn kề và miền bán kính thiết kế R từ 0,10 m đến 1,50 m. Ứng viên không được cắt quá chiều dài hình học sẵn có. Sau khi có d , sáu control point được tạo, đường Bézier được lấy mẫu khoảng 0,02 m và mọi đại lượng hình học được tính trên chuỗi mẫu.

6.4 Ràng buộc cứng của một ứng viên

Một ứng viên bị loại ngay nếu có NaN hoặc Inf, đạo hàm suy biến, đoạn chuyển động dài 0, đối dấu κ ngoài ý muốn, buộc bánh trong đảo chiều trong transition tiến, không tồn tại vận tốc dương thỏa các giới hạn, time-parameterization không hội tụ, center cost vượt ngưỡng, swept footprint chạm lethal hoặc unknown không cho phép, trim chồng góc kế tiếp, hoặc full path cuối không giữ start và goal.

Cách tổ chức này rất quan trọng về mặt nghiên cứu. An toàn là hard constraint (ràng buộc bắt buộc), không phải một số hạng nhỏ trong objective. Một ứng viên va chạm không thể được “bù” bằng việc có E_k thấp hoặc chiều dài ngắn.

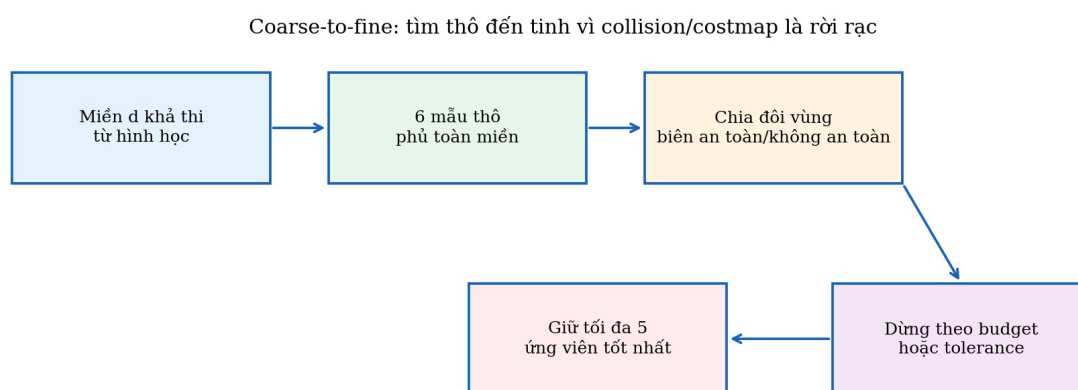
$$\bar{v} = \min(v_{\max}, \omega_{\max}/|k|, \sqrt{(a_{y,\max}/|k|)}, v_{w,\max} / \max(|1-L_k/2|, |1+L_k/2|))$$

Trong đó: $\bar{v}$ là trần vận tốc cục bộ; $v_{\max}$ là trần tuyến tính; $\omega_{\max}$ là trần vận tốc góc; $a_{y,\max}$ là trần gia tốc ngang; $v_{w,\max}$ là trần vận tốc mỗi bánh.

Ý nghĩa và lý do sử dụng: Lấy giá trị nhỏ nhất bảo đảm cùng lúc không vượt giới hạn thân xe, quay, gia tốc ngang và bánh.

Phương trình trên là trần vận tốc cục bộ của transition. Nếu không có một giá trị v dương đủ lớn để đi qua transition trong các giới hạn này thì ứng viên bị loại trước khi được so sánh objective.

6.5 Tìm kiếm thích nghi coarse-to-fine



Luồng tìm kiếm từ miền d rộng đến tối đa năm ứng viên an toàn có objective tốt.

Costmap và collision checker là rời rạc. Chỉ cần thay d rất nhỏ, một góc footprint có thể chuyển sang ô khác và trạng thái từ an toàn thành va chạm. Objective vì thế không trơn và không chắc đơn đỉnh. Gradient descent không phải lựa chọn đáng tin cậy.

Adaptive search bắt đầu bằng sáu mẫu phủ toàn miền. Sau đó nó ưu tiên chia đôi interval có biên feasible–infeasible hoặc safe–unsafe, bởi đây thường là nơi hình học thay đổi ý nghĩa. Tiếp theo thuật toán tinh chỉnh quanh objective tốt nhất và vùng objective còn biến đổi. Mỗi góc có tối đa 20 lần đánh giá, dừng sớm khi độ phân giải R đạt 0,01 m hoặc objective thay đổi dưới 0,01. Kết quả được xếp hạng ổn định và giữ tối đa năm ứng viên G^2 cùng một trạng thái Pivot.

6.6 Hàm mục tiêu

Mục tiêu hình học phải ít phụ thuộc mật độ lấy mẫu. Risk được chuẩn hóa từ peak cost, angular từ vận tốc góc cực đại tương đối, còn energy từ E_k . Các ứng viên đã vi phạm an toàn bị loại trước, nên objective chỉ dùng để xếp hạng những ứng viên hợp lệ.

$$\text{risk} = \min(1, \text{peak_cost} / 252)$$

Trong đó: risk là rủi ro chuẩn hóa 0..1; peak_cost là cost lớn nhất trên ứng viên; 252 là mức cost tham chiếu trước lethal.

Ý nghĩa và lý do sử dụng: Thành phần này chỉ xếp hạng các ứng viên đã qua kiểm tra an toàn; nó không thay collision checker.

$$\text{angular} = \min(1, |\omega|_{\text{peak}} / \omega_{\text{max}})$$

Trong đó: angular là mức sử dụng vận tốc góc chuẩn hóa; $|\omega|_{\text{peak}}$ là vận tốc góc cực đại dự kiến; ω_{max} là giới hạn hệ thống.

Ý nghĩa và lý do sử dụng: Ứng viên cần dùng gần hết khả năng quay được đánh giá kém hơn vì ít biên điều khiển.

$$\text{energy} = E_k / (E_k + 1,0 \text{ m}^{-1})$$

Trong đó: energy là E_k đã chuẩn hóa về 0..1; E_k là năng lượng độ cong.

Ý nghĩa và lý do sử dụng: Phép chuẩn hóa tránh một đại lượng có thang quá lớn lấn át các thành phần khác trong objective.

$$J = 0,15 \cdot \text{risk} + 0,10 \cdot \text{angular} + 0,75 \cdot \text{energy}$$

Trong đó: J là objective cần giảm; 0,15;0,10;0,75 là các trọng số; risk,angular,energy là ba đại lượng đã chuẩn hóa.

Ý nghĩa và lý do sử dụng: Trọng số hiện tại ưu tiên giảm năng lượng độ cong nhưng vẫn giữ thông tin proximity và mức sử dụng vận tốc góc. An toàn được xử lý trước bằng hard constraint.

Trọng số 0,75 cho energy phản ánh mục tiêu chính của lỗi Pivot- G^2 là giảm độ gắt tổng thể. Tuy nhiên candidate (ứng viên) còn phải nằm trong time gate so với Pivot hoặc ứng viên nhanh nhất, tránh chọn một đường quá mềm nhưng tốn thời gian không hợp lý.

6.7 Quy hoạch động giữa nhiều góc

Nếu tối ưu từng góc độc lập theo kiểu greedy thì hai góc liên tiếp có thể cùng chọn d lớn và cắt chồng lên cùng một đoạn thẳng. Vì vậy toàn bộ đường được biểu diễn thành một chuỗi lớp trạng thái; mỗi lớp là tập candidate của một góc. Hai trạng thái chỉ được nối với nhau khi tổng trim cộng margin không vượt chiều dài đoạn chung.

$$d_{i,a} + d_{i+1,\beta} + m_i \leq L_i$$

Trong đó: $d_{i,a}$ và $d_{i+1,\beta}$ là trim của hai góc kề; m_i là khoảng dự phòng; L_i là chiều dài đoạn chung.

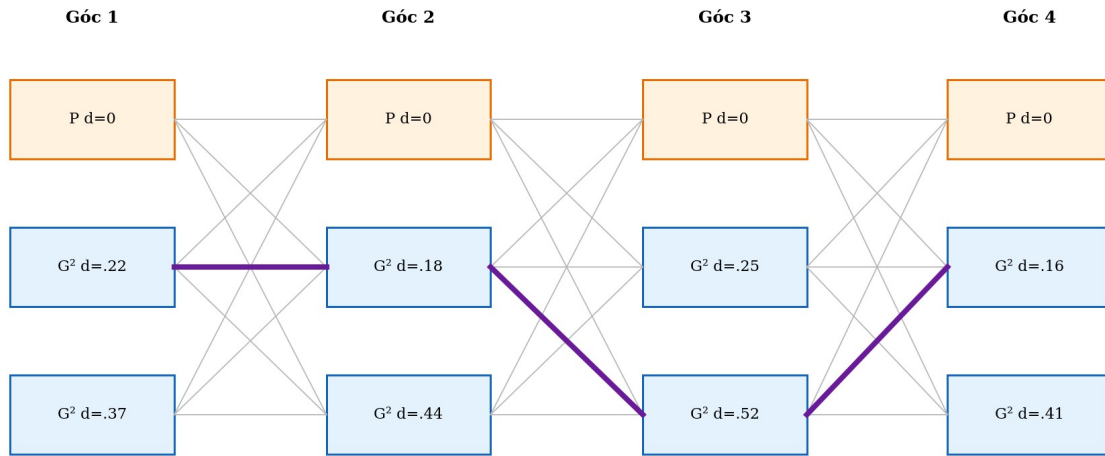
Ý nghĩa và lý do sử dụng: Bất đẳng thức ngăn hai transition ăn chồng lên cùng một đoạn thẳng.

$$D_i(k) = J_i(k) + \min_j \text{tương thích với } k D_{i-1}(j)$$

Trong đó: $D_i(k)$ là cost tích lũy tốt nhất tới trạng thái k ở góc i; $J_i(k)$ là cost cục bộ; j là trạng thái tương thích ở góc trước.

Ý nghĩa và lý do sử dụng: Đây là truy hồi quy hoạch động giúp chọn chuỗi ứng viên toàn đường thay vì quyết định greedy từng góc.

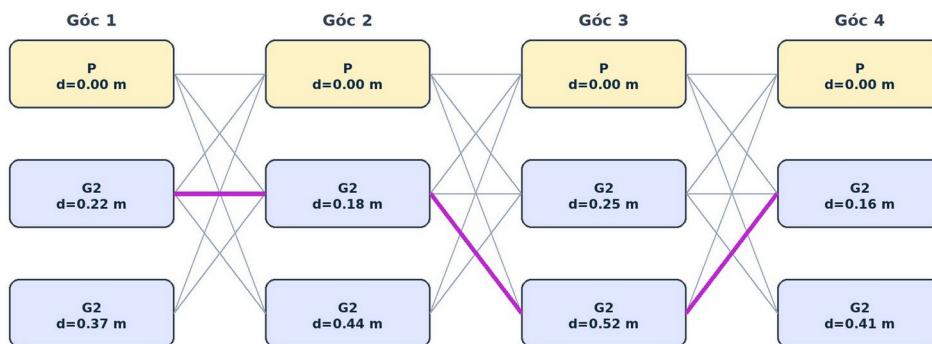
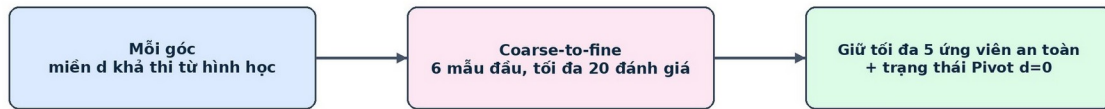
Margin mặc định là giá trị lớn nhất giữa output spacing, hai lần sample spacing và costmap resolution, hiện bằng 0,05 m. Độ phức tạp của quy hoạch động là $O(NK^2)$, với N là số góc và K là số trạng thái trung bình. Khi hai phương án bằng cost, code ưu tiên ít Pivot hơn rồi ưu tiên index nhỏ hơn để bảo đảm deterministic.



DP chỉ nối hai trạng thái khi tổng trim không vượt chiều dài đoạn chung

Quy hoạch động ghép trạng thái giữa các góc và chọn một chuỗi toàn đường không chồng trim.

TÌM KIẾM THÍCH NGHI + QUY HOẠCH ĐỘNG TOÀN ĐƯỜNG



DP chỉ nối hai trạng thái nếu $d_i + d_{i+1} + \text{margin} \leq \text{chiều dài đoạn chung}$; đường màu tím là chuỗi có tổng cost thấp nhất.

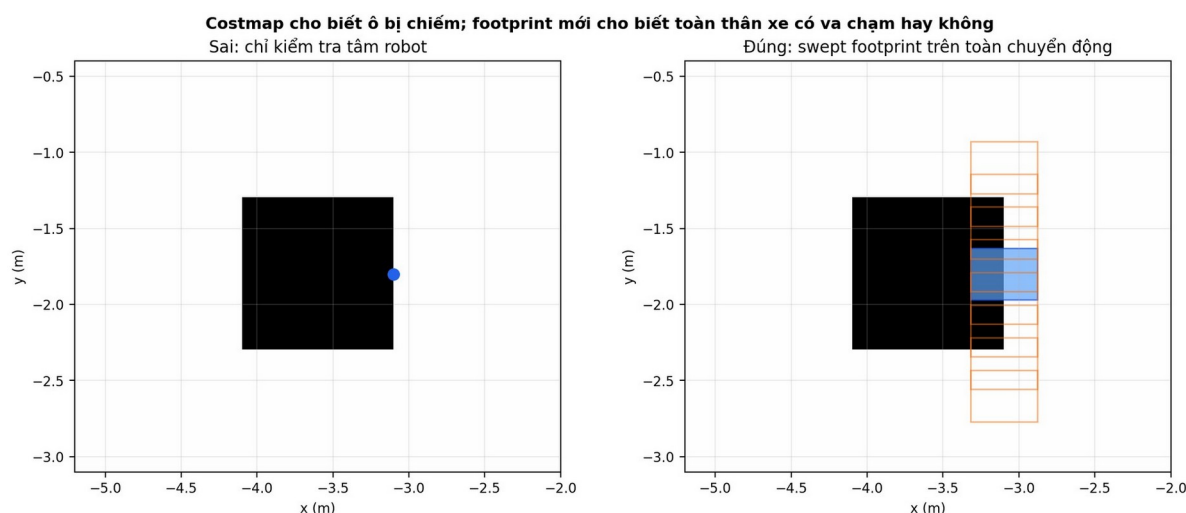
Sơ đồ tìm kiếm thích nghi và quy hoạch động được dùng trong báo cáo dữ liệu dự án.

6.8 Khâu toàn đường và kiểm tra swept footprint

Sau backtracking, các đoạn thẳng còn lại, transition Bézier và marker Pivot được khâu thành một path hoàn chỉnh. Path được resample khoảng 0,05 m, orientation được gán theo tiếp

tuyến, endpoint được sửa về đúng start và goal, sau đó kiểm tra lại duplicate, NaN, động học và va chạm.

Không được chỉ kiểm tra tâm robot. Một điểm tâm nằm trong ô trống vẫn có thể khiến góc thân xe chạm kệ. Swept footprint là hợp của footprint khi robot quét dọc toàn đoạn chuyển động. Mỗi pose mẫu mang orientation riêng, footprint chữ nhật được xoay và chiếu xuống costmap. Nếu bất kỳ cấu hình nào giao lethal obstacle thì toàn bộ shortcut hoặc transition bị loại.



Minh họa lý do kiểm tra tâm robot là chưa đủ và mọi shortcut hoặc transition phải kiểm tra swept footprint.

6.9 Công an toàn Adaptive Hybrid

Pure Pivot- G^2 có thể cho E_k rất thấp nhưng điều đó chưa bảo đảm controller thực hiện tốt trong một hành lang sát vật cản. Trong một số trường hợp Nav2 Simple tạo ra clearance tốt hơn hoặc phù hợp hơn với dự báo collision của RPP. Vì vậy phương pháp hoàn chỉnh dùng Simple làm nhánh bảo thủ và chỉ nhận Pivot- G^2 khi có lợi ích an toàn đủ rõ hoặc khi Simple không an toàn.

Hai gate hiện tại so sánh chênh lệch cost và ngân sách năng lượng độ cong. Nếu cả Simple và Pivot an toàn nhưng Pivot không tạo đủ safety gain, hệ thống giữ Simple. Nếu Simple không an toàn còn Pivot an toàn, hệ thống chọn Pivot. Nếu cả hai không an toàn nhưng Raw vẫn an toàn, hệ thống fallback Raw. Nếu Raw, Simple và Pivot đều không an toàn, smoother trả fail thay vì xuất một path nguy hiểm.

$$\Delta cost = costSimple - costPivot \geq 20$$

Trong đó: $\Delta cost$ là lợi ích proximity của Pivot so với Simple; 20 là ngưỡng safety gain.

Ý nghĩa và lý do sử dụng: Hybrid chỉ chọn Pivot khi nhánh đó cải thiện khoảng gần vật cản đủ rõ, tránh đổi đường chỉ vì một khác biệt cost nhỏ.

$$EPivot \leq 2(ESimple + 0,25)$$

Trong đó: $EPivot$ và $ESimple$ là năng lượng độ cong của hai nhánh; 0,25 là hệ số ổn định; 2 là ngân sách cho phép.

Ý nghĩa và lý do sử dụng: Điều kiện ngăn Pivot đạt safety gain nhưng làm độ cong xấu quá mức so với Simple.

Tình trạng	Đường được chọn	Lý do
Simple và Pivot an toàn, Pivot qua hai gate	Pivot- G^2	Có lợi proximity mà không tăng E_k quá ngân sách.
Cả hai an toàn nhưng Pivot không qua gate	Simple	Mặc định bảo thủ.
Simple không an toàn, Pivot an toàn	Pivot- G^2	Nhánh simple_unsafe.
Pivot không an toàn, Simple an toàn	Simple	Nhánh pivot_unsafe.
Cả hai không an toàn, Raw an toàn	Raw	Raw fallback.
Raw, Simple, Pivot đều không an toàn	Fail	Không xuất đường nguy hiểm.

VII, Profile vận tốc và điều khiển vòng kín

7.1 Trần vận tốc theo độ cong

Path sau smoother vẫn chưa có thời gian. Tại mỗi mẫu, trần vận tốc được tính từ bốn nguồn chính: vận tốc tuyến tính toàn cục, gia tốc ngang, vận tốc góc và vận tốc bánh. Đường thẳng có κ gần 0 nên chủ yếu bị giới hạn bởi v_{max} và bánh. Khi $|\kappa|$ tăng, ba trần còn lại giảm.

$$\bar{v}(s) = \min[v_{max}, \sqrt{(a_{y,max}/|\kappa|)}, \omega_{max}/|\kappa|, v_{w,max}/(1+L|\kappa|/2)]$$

Trong đó: $\bar{v}(s)$ là trần vận tốc tại tiến độ s ; κ là độ cong tại s ; các tham số max là giới hạn thân xe và bánh.

Ý nghĩa và lý do sử dụng: Đường cong càng gắt thì ba trần phụ thuộc κ càng giảm, buộc robot tự chạy chậm khi vào cua.

7.2 Khoảng chuyển vận tốc giới hạn jerk

Chỉ giới hạn gia tốc vẫn cho phép gia tốc nhảy tức thời. Jerk là tốc độ thay đổi của gia tốc. Hệ thống dùng profile kiểu S-curve đơn giản để tính khoảng cách tối thiểu cần thiết khi chuyển từ v_0 sang v_1 . Nếu Δv nhỏ, profile gia tốc có dạng tam giác; nếu Δv lớn, profile có thêm đoạn giữ gia tốc.

$$\text{Nếu } \Delta v \leq a^2/j: \quad S = (v_0 + v_1)\sqrt{(\Delta v/j)}$$

Trong đó: Δv là độ thay đổi vận tốc; a là giới hạn gia tốc; j là giới hạn jerk; S là khoảng cách cần để chuyển vận tốc; v_0, v_1 là vận tốc đầu/cuối.

Ý nghĩa và lý do sử dụng: Đây là trường hợp profile gia tốc tam giác: chưa kịp đạt a tối đa thì đã phải giảm gia tốc.

$$\text{Nếu } \Delta v > a^2/j: \quad S = 0,5(v_0 + v_1)(\Delta v/a + a/j)$$

Trong đó: các ký hiệu giống công thức trên.

Ý nghĩa và lý do sử dụng: Đây là trường hợp profile có đoạn giữ gia tốc cực đại. Code đảo công thức bằng tìm kiếm nhị phân để suy ra vận tốc tối đa từ khoảng cách còn lại.

Trong code, phương trình được đảo bằng tìm kiếm nhị phân. Biết khoảng cách còn lại đến một cap thấp hơn, thuật toán tìm vận tốc lớn nhất tại điểm hiện tại mà robot vẫn kịp phanh hoặc tăng tốc trong giới hạn.

7.3 Hai lượt truyền và gia tốc góc

Backward pass truyền giới hạn tương lai về phía trước của đường, làm robot giảm tốc sớm trước cong, Pivot hoặc goal. Forward pass truyền giới hạn quá khứ về phía sau, ngăn robot tăng tốc ngay sau cong nhanh hơn cơ cấu cho phép. Hai pass được lặp đến hội tụ vì một cap mới sinh ra ở pass này có thể làm thay đổi pass kia.

Sau đó các cặp mẫu tiếp tục bị scale cho đến khi $|\Delta(v\kappa)|/\Delta t$ không vượt α_{max} . Đây là cách liên kết profile tuyến tính với khả năng thay đổi vận tốc góc của robot.

7.4 Phép chiếu tiến độ có xét hướng

Controller cần biết robot đang ở vị trí nào dọc path. Nếu chỉ chọn điểm gần nhất theo khoảng cách Euclidean, robot có thể nhảy sang một nhánh khác khi đường tự cắt hoặc hai đoạn song song nằm gần nhau. Vì vậy score của projection gồm cả sai số vị trí và sai số hướng.

$$\text{score} = e^2_{xy} + (0,20 \cdot e_{\psi})^2$$

Trong đó: e_{xy} là sai số vị trí đến đường; e_{ψ} là sai số hướng; 0,20 m/rad là hệ số quy đổi hướng sang thang vị trí.

Ý nghĩa và lý do sử dụng: Projection xét cả khoảng cách và hướng để tránh chọn nhầm một nhánh gần về vị trí nhưng ngược chiều.

Tìm kiếm chỉ diễn ra trong cửa sổ 0,25 m phía sau và 0,80 m phía trước hint. Tiến độ chỉ được lùi tối đa 0,03 m. Khi hai điểm có score bằng nhau, hệ thống chọn s lớn hơn. Quy tắc này làm projection có tính nhân quả và giảm nguy cơ nhảy ngược.

7.5 RPP và giảm tốc theo sai số bám

Pure Pursuit chọn một carrot phía trước robot. Trong frame thân xe, nếu carrot có tọa độ (x_L , y_L) và lookahead ℓ thì độ cong xấp xỉ $2y_L/\ell^2$. RPP của Nav2 bổ sung giảm tốc theo độ cong, cost và thời gian dự báo va chạm.

Wrapper của dự án lấy giá trị nhỏ nhất giữa lệnh RPP và các cap. Khi cross-track error, heading error hoặc angular-tracking error đi từ ngưỡng mềm đến ngưỡng cứng, cap giảm bằng smoothstep. Nhờ đó robot không chuyển đột ngột từ chạy nhanh sang chạy rất chậm chỉ vì sai số vừa vượt một ngưỡng.

$$r = (|e| - e_{\text{soft}})/(e_{\text{hard}} - e_{\text{soft}}), \quad h(r) = 3r^2 - 2r^3$$

Trong đó: e là sai số đang xét; e_{soft} và e_{hard} là hai ngưỡng; r là sai số chuẩn hóa; $h(r)$ là hàm smoothstep.

Ý nghĩa và lý do sử dụng: Smoothstep chuyển cap trơn từ mức bình thường sang mức giảm tốc, không tạo góc gãy tại hai ngưỡng.

$$v_{\text{cap}} = (1-h)v_{\text{max}} + h \cdot v_{\text{min}}$$

Trong đó: v_{cap} là trần vận tốc do sai số bám; v_{max} và v_{min} là hai mức biên; h là hệ số 0..1.

Ý nghĩa và lý do sử dụng: Sai số càng gần hard threshold thì h càng gần 1 và vận tốc bị kéo về v_{min} .

7.6 Bộ tạo lệnh jerk-limited và safety override

$$a^* = \text{clamp}((v_{\text{target}} - v_{\text{prev}})/\Delta t, -a_{\text{dec}}, a_{\text{acc}})$$

Trong đó: a^* là gia tốc mong muốn; v_{target} là vận tốc mục tiêu; v_{prev} là vận tốc trước; Δt là chu kỳ; $a_{\text{dec}}, a_{\text{acc}}$ là giới hạn giảm/tăng tốc.

Ý nghĩa và lý do sử dụng: Bước đầu giới hạn gia tốc theo khả năng phanh và tăng tốc của robot.

$$acmd = \text{clamp}(a^*, a_{\text{prev}} - j\Delta t, a_{\text{prev}} + j\Delta t)$$

Trong đó: $acmd$ là gia tốc được phát; a_{prev} là gia tốc trước; j là jerk tối đa.

Ý nghĩa và lý do sử dụng: Bước thứ hai không cho gia tốc thay đổi nhanh hơn $j\Delta t$ trong một chu kỳ.

$$vcmd = \min(v_{\text{target}}, \max(0, v_{\text{prev}} + acmd\Delta t))$$

Trong đó: $vcmd$ là vận tốc lệnh cuối; các ký hiệu còn lại như trên.

Ý nghĩa và lý do sử dụng: Bước cuối tích phân gia tốc để tạo vận tốc và bảo đảm không vượt target hoặc xuống dưới 0 trong chuyển động tiến.

Nếu một safety cap đột ngột thấp hơn vận tốc jerk-limited cho phép, an toàn được quyền thắng ngay. Mẫu đó được gán `safety_override` để dữ liệu benchmark không giả vờ rằng jerk vật lý luôn được giữ trong tình huống cần phanh khẩn.

7.7 Neo start và goal liên tục trước khi đánh giá hoặc làm mượt

Một lỗi nhỏ về biểu diễn có thể làm sai toàn bộ quá trình đánh giá. Global planner làm việc trên occupancy grid nên pose đầu hoặc cuối của `nav_msgs/Path` có thể bị đưa về tâm ô lưới gần nhất, trong khi pose thực của robot và goal người dùng chọn là tọa độ liên tục trong frame map. Nếu giữ nguyên tâm ô như thế đó là start thật, controller sẽ nhìn thấy một sai số ngang giả ngay từ lúc bắt đầu. Khi robot vừa ra khỏi đường cong hoặc đang căn goal, sai số giả này còn có thể làm profile vận tốc phanh hoặc tăng tốc sai thời điểm.

$$\delta_{\text{grid,max}} = \sqrt{(\text{rmap}/2)^2 + (\text{rmap}/2)^2} = \text{rmap}/\sqrt{2}$$

Trong đó: $\delta_{\text{grid,max}}$ là độ lệch lớn nhất giữa một điểm liên tục và tâm ô chứa điểm đó; rmap là độ phân giải map, hiện bằng 0,05 m/ô.

Ý nghĩa và lý do sử dụng: với $\text{rmap} = 0,05$ m, độ lệch cực đại chỉ do lượng tử hóa đã bằng 0,035355 m, tức khoảng 3,54 cm. Con số này lớn hơn sai số bám trung bình mà hệ thống đang cố giảm, vì vậy không thể xem nó là nhiễu không đáng kể.

Hợp đồng path mới thực hiện hai bước. Trước hết kiểm tra khoảng cách từ pose đầu của planner đến start liên tục và từ pose cuối đến goal liên tục. Nếu khoảng điều chỉnh không vượt 0,08 m, vị trí đầu và cuối được phục hồi đúng theo yêu cầu gốc. Orientation ở hai pose biên được lấy từ start/goal yêu cầu; controller không dùng riêng orientation đầu để suy ra tiếp tuyến mà tính lại bằng preview của selected path. Nếu phải điều chỉnh lớn hơn 0,08 m, trial bị loại thay vì âm thầm kéo path qua một vùng mà planner chưa chứng minh là an toàn.

$$p_0 \leftarrow p_{\text{start}}, \quad p_N \leftarrow p_{\text{goal}}, \quad \text{nếu } \|p_0 - p_{\text{start}}\| \leq 0,08 \text{ và } \|p_N - p_{\text{goal}}\| \leq 0,08$$

Trong đó: p_0 và p_N là hai pose biên của path; p_{start} là pose liên tục tại thời điểm lập đường; p_{goal} là pose đích do scenario hoặc RViz2 cung cấp; $\|\cdot\|$ là khoảng cách Euclidean trên mặt phẳng.

Ý nghĩa và lý do sử dụng: giới hạn 0,08 m đủ lớn để sửa lượng tử hóa của map 0,05 m nhưng vẫn đủ nhỏ để phát hiện một path bị lệch frame, bị dùng nhầm start, hoặc bị ghép sai sau smoother.

Bước neo được áp dụng cho cả raw path do planner trả về và selected path sau khi smoother lựa chọn. Bốn đại lượng `planner_start_anchor_adjustment_m`, `planner_goal_anchor_adjustment_m`, `selected_start_anchor_adjustment_m` và `selected_goal_anchor_adjustment_m` được ghi vào kết quả. Cách làm này bảo đảm tám smoother được chấm trên cùng một miền start-goal, đồng thời không để chính bước làm mượt âm thầm di chuyển hai điều kiện biên.

7.8 Xác định hướng xuất phát theo hai tầng

Hướng đặt robot ban đầu không nên được lấy đơn giản bằng đường thẳng nối start đến goal. Trong map có kệ hoặc tường, bearing trực tiếp có thể chỉ vào vật cản dù đường hợp lệ phải rẽ sang một hành lang khác. Nếu Gazebo spawn robot theo bearing sai, controller sẽ mất nhiều thời gian quay ngược; nếu vừa quay vừa tiến, bánh dễ kéo xe lệch khỏi ray tham chiếu ngay trước góc đầu tiên.

$$\psi_{\text{direct}} = \text{atan2}(y_{\text{goal}} - y_{\text{start}}, x_{\text{goal}} - x_{\text{start}})$$

Trong đó: ψ_{direct} là hướng trực tiếp từ start đến goal; x_{start} , y_{start} là tọa độ start; x_{goal} , y_{goal} là tọa độ goal; atan2 giữ đúng góc phần tư và trả yaw trong radian.

Ý nghĩa và lý do sử dụng: ψ_{direct} chỉ là ứng viên đầu tiên, không phải đáp án bắt buộc. Nó được chấp nhận khi hướng đó có khoảng thăm dò footprint an toàn hoặc khi line-of-sight trên occupancy grid không đi qua ô chiếm dụng.

Tầng thứ nhất diễn ra trước khi spawn hoặc trước khi gửi trial. File YAML của map cung cấp resolution, origin và đường dẫn PGM. Tọa độ world được đổi về tọa độ cục bộ của map, sau đó lấy chỉ số cột và hàng bằng phép floor. Thuật toán không chỉ kiểm tra một tia từ tâm; nó thăm dò theo các hướng ứng viên với bề rộng footprint để tránh chọn một hướng mà tâm đi qua được nhưng góc thân xe chạm kệ.

$$c = \text{floor}(x_{\text{local}}/r_{\text{map}}), \quad r = H - 1 - \text{floor}(y_{\text{local}}/r_{\text{map}})$$

Trong đó: c và r là chỉ số cột, hàng của ảnh PGM; H là chiều cao ảnh; x_{local} , y_{local} là tọa độ sau khi trừ origin và quay về hệ map; phép $H - 1 - \cdot$ xuất hiện vì trục hàng của ảnh tăng từ trên xuống, trong khi trục y của bản đồ tăng từ dưới lên.

Nếu bearing trực tiếp không an toàn, tầng này ước lượng một tuyến grid route có ưu tiên clearance và lấy hướng của đoạn đầu đủ dài. Tên nguồn quyết định, ví dụ `map_aware_grid_route` hoặc `direct_bearing`, được ghi vào `initial_heading_source`. Khi scenario đã cung cấp yaw tường minh, yaw đó được giữ vì nó là một phần của điều kiện thử, không bị thuật toán tự ý thay thế.

Tầng thứ hai diễn ra sau khi planner và smoother đã tạo selected path. Controller không tin tuyệt đối orientation ở pose đầu, vì một planner 2D có thể để quaternion mặc định hoặc

orientation chưa phản ánh đúng tiếp tuyến. Nó lấy một điểm preview cách đầu đường 0,30 m theo chiều dài cung rồi tính hướng dây cung từ pose đầu đến điểm đó.

$$\psi_{\text{preview}} = \text{atan2}(y_{\text{preview}} - y_0, x_{\text{preview}} - x_0), \quad s_{\text{preview}} = 0,30 \text{ m}$$

Trong đó: ψ_{preview} là hướng controller cần căn; $p_0 = (x_0, y_0)$ là đầu selected path; p_{preview} là điểm nội suy tại tiến độ 0,30 m. Khoảng preview đủ dài để không bị chi phối bởi hai waypoint gần như trùng nhau nhưng vẫn ngắn hơn phần lớn đoạn vào hành lang.

Nếu $|\text{wrap}(\psi_{\text{preview}} - \psi_{\text{robot}})|$ nhỏ hơn 0,15 rad, robot được phép vào bám đường ngay. Nếu lớn hơn hoặc bằng ngưỡng này, trạng thái initial_alignment được kích hoạt: vận tốc tuyến tính bằng 0 và robot quay tại chỗ. Trạng thái chỉ được nhả khi sai số nhỏ hơn 0,035 rad, đồng thời vận tốc tuyến tính và góc đều đã nằm trong ngưỡng dừng. Hai ngưỡng khác nhau tạo hysteresis, tránh trạng thái bật–tắt liên tục quanh một giá trị duy nhất.

7.9 Bao phanh góc theo khả năng giảm tốc thực

Giới hạn gia tốc lệnh $1,2 \text{ rad/s}^2$ chỉ quy định mỗi chu kỳ được thay đổi lệnh bao nhiêu; nó không chứng minh thân xe thực sự giảm vận tốc góc với đúng giá trị đó. Trace Gazebo cho thấy khi phải đảo chiều quay trong initial alignment, mức giảm tốc góc hiệu dụng chỉ khoảng $0,18 \text{ rad/s}^2$. Nếu tiếp tục dùng một lệnh tốc độ góc gần hằng số đến sát target rồi mới hạ, robot sẽ đi qua hướng cần đạt và quay ngược lại. Quá trình này lặp thành counter-rotation, làm mất thời gian và có thể kéo robot lệch đường khi chuyển sang vận tốc tuyến tính.

$$\omega_{\text{brake}} = \min[\omega_{\text{max}}, \sqrt{(2\alpha_{\text{eff}} \cdot \max(|e\psi| - \psi_{\text{tol}}, 0))}]$$

Trong đó: ω_{brake} là trần vận tốc góc còn được phép phát; $\omega_{\text{max}} = 0,70 \text{ rad/s}$ là trần quay; $\alpha_{\text{eff}} = 0,18 \text{ rad/s}^2$ là giảm tốc góc hiệu dụng đo từ Gazebo; $e\psi$ là sai số heading có dấu; $\psi_{\text{tol}} = 0,015 \text{ rad}$ là deadband của Pivot.

Ý nghĩa và lý do sử dụng: công thức được suy ra từ quan hệ động học $\omega^2 = 2\alpha\theta$. Phần $|e\psi| - \psi_{\text{tol}}$ là góc thật sự còn được dùng để phanh; deadband được trừ trước để robot không cố dừng tại một điểm toán học không có bề rộng.

$$\omega_{\text{des}} = \text{sign}(e\psi) \cdot \min(K\psi |e\psi|, \omega_{\text{brake}})$$

Trong đó: ω_{des} là vận tốc góc mong muốn; $K\psi$ là hệ số phản hồi theo sai số; $\text{sign}(e\psi)$ chọn chiều quay. Sau đó lệnh thực vẫn bị giới hạn bởi $|\omega_{\text{cmd}} - \omega_{\text{measured}}| \leq \alpha_{\text{cmd}}\Delta t$ với $\alpha_{\text{cmd}} = 1,2 \text{ rad/s}^2$.

Ý nghĩa và lý do sử dụng: luật tách ba vai trò. $K\psi$ tạo phản hồi gần target; bao căn bậc hai quyết định lúc phải phanh theo động học; α_{cmd} giới hạn độ dốc của lệnh theo chu kỳ. Không nên gộp ba đại lượng này thành một tham số vì chúng mô tả ba hiện tượng khác nhau.

Ví dụ với $|e\psi| = 0,961 \text{ rad}$, $\psi_{\text{tol}} = 0,015 \text{ rad}$ và $\alpha_{\text{eff}} = 0,18 \text{ rad/s}^2$, trần phanh bằng khoảng $0,584 \text{ rad/s}$, thấp hơn $\omega_{\text{max}} = 0,70 \text{ rad/s}$. Robot bắt đầu giảm tốc sớm thay vì giữ $0,70 \text{ rad/s}$ đến gần đích. Trên narrow_aisles, số mẫu initial_alignment giảm từ 209 xuống 93 và thời gian hoàn thành giảm từ 74,721 s xuống 67,104 s.

7.10 Biên vận tốc bằng không và cách báo jerk trung thực

Bộ tạo S-curve hoạt động tốt trong miền $v > 0$, nhưng tại thời điểm dừng có một biên khả thi đặc biệt. Nếu gia tốc trước đang âm và tốc độ còn lại quá nhỏ, phép tích phân $v_{prev} + a_{cmd}\Delta t$ có thể cho kết quả âm trước khi jerk kịp kéo gia tốc về 0. Robot không được phép chạy với độ lớn vận tốc âm trong nhánh chỉ chuyển động tiến, vì vậy phải clip tại 0. Mẫu clip này là một safety override thật, không phải một mẫu nominal jerk.

$$\begin{aligned}v_{unconstrained} &= v_{prev} + a_{cmd}\Delta t \\v_{shape} &= \max(0, v_{unconstrained})\end{aligned}$$

Trong đó: $v_{unconstrained}$ là kết quả tích phân trước ràng buộc; v_{shape} là vận tốc không âm; a_{cmd} là gia tốc sau giới hạn jerk.

Nếu $v_{unconstrained} < 0$, cờ `zero_speed_override` được bật. Nếu một trần an toàn mới làm v_{target} nhỏ hơn v_{shape} , cờ `upper_cap_override` được bật. Hai trường hợp đều được gộp vào `safety_override`. Jerk đo từ chênh lệch gia tốc vẫn được lưu nguyên giá trị để dữ liệu trung thực, nhưng không được tính vào `adaptive_speed_nominal_max_abs_jerk`.

$$safety_override = upper_cap_override \vee zero_speed_override$$

Trong đó: ký hiệu $\vee$ nghĩa là phép OR logic. Chỉ cần một điều kiện an toàn xảy ra thì mẫu đó không còn thuộc miền S-curve danh nghĩa.

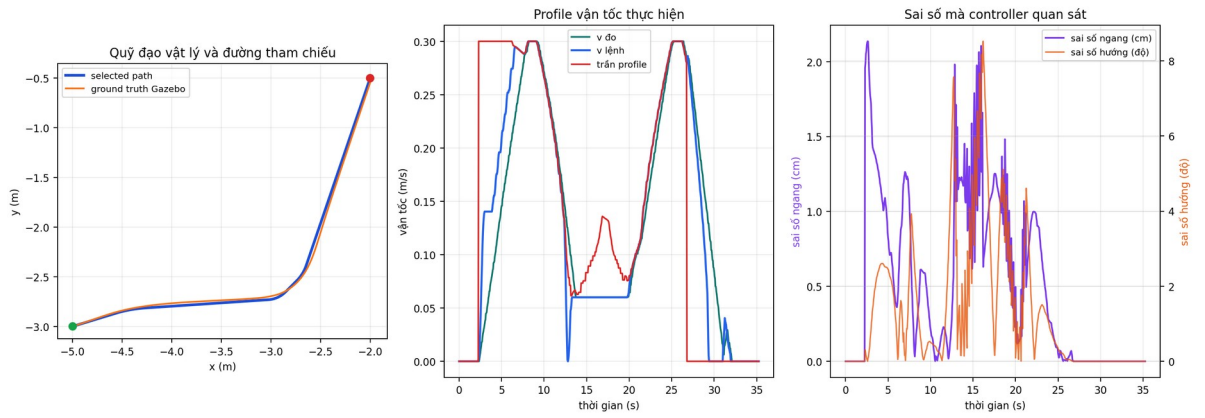
Khi v_{shape} đã về 0, trạng thái gia tốc nội bộ được reset về 0. Nếu giữ lại một gia tốc âm ngoài miền hợp lệ, chu kỳ sau sẽ tưởng robot vẫn đang phanh và có thể tạo một xung jerk giả khi tăng tốc lại. Nhờ phân loại này, giới hạn nominal jerk $0,9 \text{ m/s}^3$ được kiểm tra đúng ngữ nghĩa, còn các clip vì an toàn vẫn hiện rõ trong trace thay vì bị che đi bởi một thống kê đã lọc.

7.11 Thực thi Pivot và terminal servo

Marker Pivot được biểu diễn bằng hai pose cùng vị trí nhưng khác yaw. Khi nhận marker, controller giảm v về 0, quay với tốc độ tối đa $0,70 \text{ rad/s}$ và dùng deadband $0,015 \text{ rad}$. Sau khi yaw đạt yêu cầu, controller mới tiếp tục bám đoạn kế tiếp.

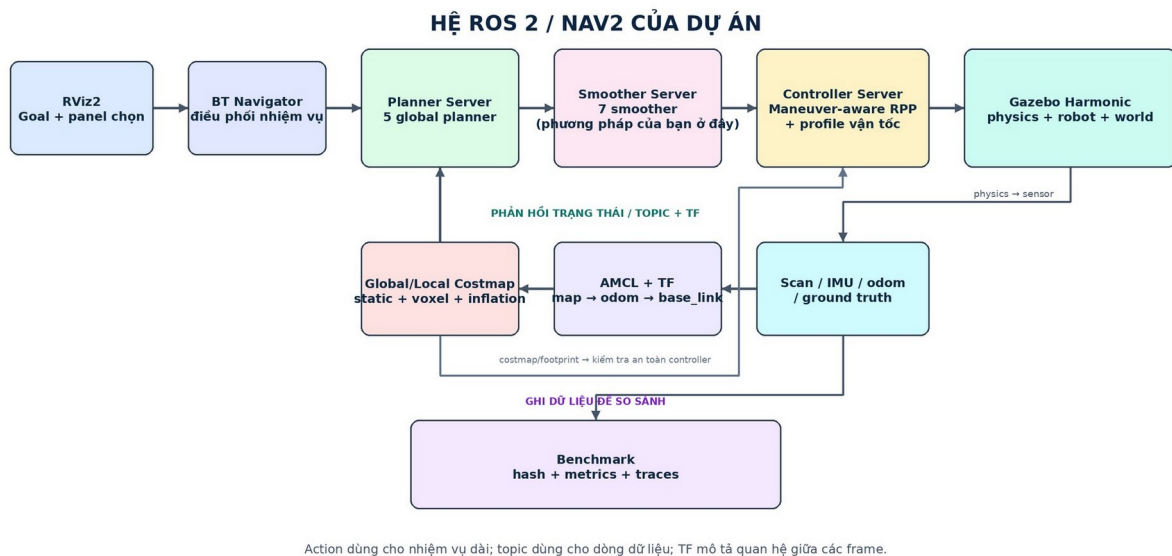
Tại goal, robot phanh vào staging radius. Vị trí đích được chốt trong odom để tránh target dịch theo nhiễu TF. Controller cho phép servo vị trí tiến hoặc lùi với tốc độ thấp, rồi mới căn yaw theo TF mới nhất. Nếu sai số vị trí tăng quá $0,04 \text{ m}$ trong lúc quay, hệ thống quay lại pha lấy XY. Cấu trúc này giải quyết lỗi robot quay đúng yaw nhưng trượt khỏi vùng goal.

Trace cuối 26/07/2026 — research_warehouse/lower_left_diagonal
t = 35.313 s; GT RMSE = 2.148 cm; GT max = 3.519 cm



Trace cuối ngày 26/07/2026 trên research_warehouse/lower_left_diagonal: selected path, ground truth Gazebo, profile vận tốc và sai số controller. Các số trong tiêu đề hình được đọc trực tiếp từ file final JSON.

VIII, Thuật toán được đặt vào ROS 2 / Nav2 như thế nào?



Kiến trúc node và luồng dữ liệu của dự án trong ROS 2/Nav2.

8.1 Topic, service, action và QoS

Topic dùng cho dòng dữ liệu liên tục như `/scan`, `/odom`, `/cmd_vel` và `telemetry`. Service dùng cho yêu cầu ngắn có request–response. Action dùng cho nhiệm vụ kéo dài, có feedback và có thể hủy, ví dụ `ComputePathToPose`, `SmoothPath` và `FollowPath`. QoS quy định reliable hay best-effort, độ sâu queue và việc có giữ mẫu cuối hay không.

Trong hệ thống này, goal từ RViz2 đi vào Behavior Tree. Planner Server gọi planner plugin. Smoother Server gọi plugin Adaptive Hybrid (cơ chế lai có lựa chọn thích nghi) Pivot–G² hoặc các baseline. Controller Server chạy maneuver-aware RPP. Gazebo tích phân chuyển động từ `/cmd_vel`, xuất odometry, sensor và ground truth (trạng thái vật lý chuẩn dùng để chấm). Benchmark ghi lại hash, metric và trace.

8.2 Các frame quan trọng

Frame	Nguồn tạo	Vai trò
world	Gazebo	Hệ tuyệt đối của mô phỏng, dùng làm nguồn ground truth.
map	Map server / AMCL	Hệ bản đồ tĩnh của goal và global path.
odom	Wheel odometry	Hệ cục bộ liên tục nhưng có thể drift.
base_link	Robot model	Gốc thân xe ở cao độ tâm bánh.
base_footprint	Fixed joint	Hình chiếu base xuống mặt đất.

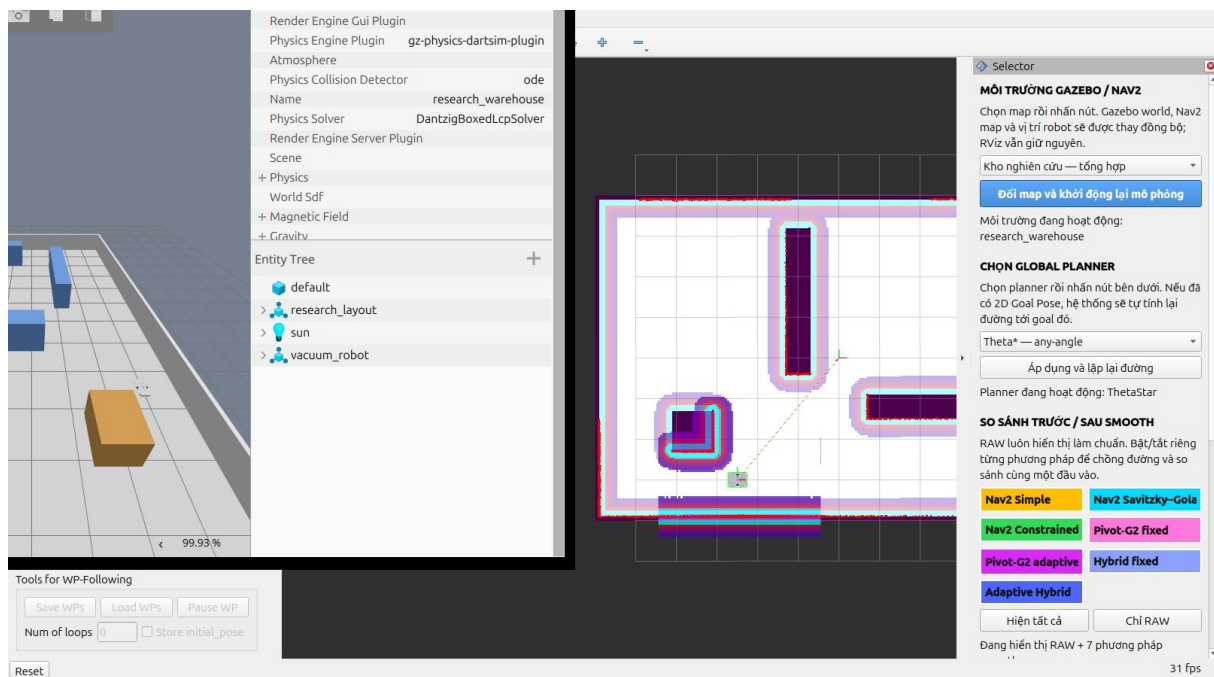
Frame	Nguồn tạo	Vai trò
laser	Robot model	Frame của RPLIDAR.
imu_link	Robot model	Frame của BNO055.

Chuỗi TF chính là map $\rightarrow$ odom $\rightarrow$ base_link. AMCL sửa quan hệ map $\rightarrow$ odom, odometry cập nhật odom $\rightarrow$ base_link. Benchmark không dùng chính TF làm ground truth cho TF. Pose vật lý được lấy độc lập từ Gazebo trong world, sau đó căn chỉnh sang map để tính tracking error và localization error riêng biệt.

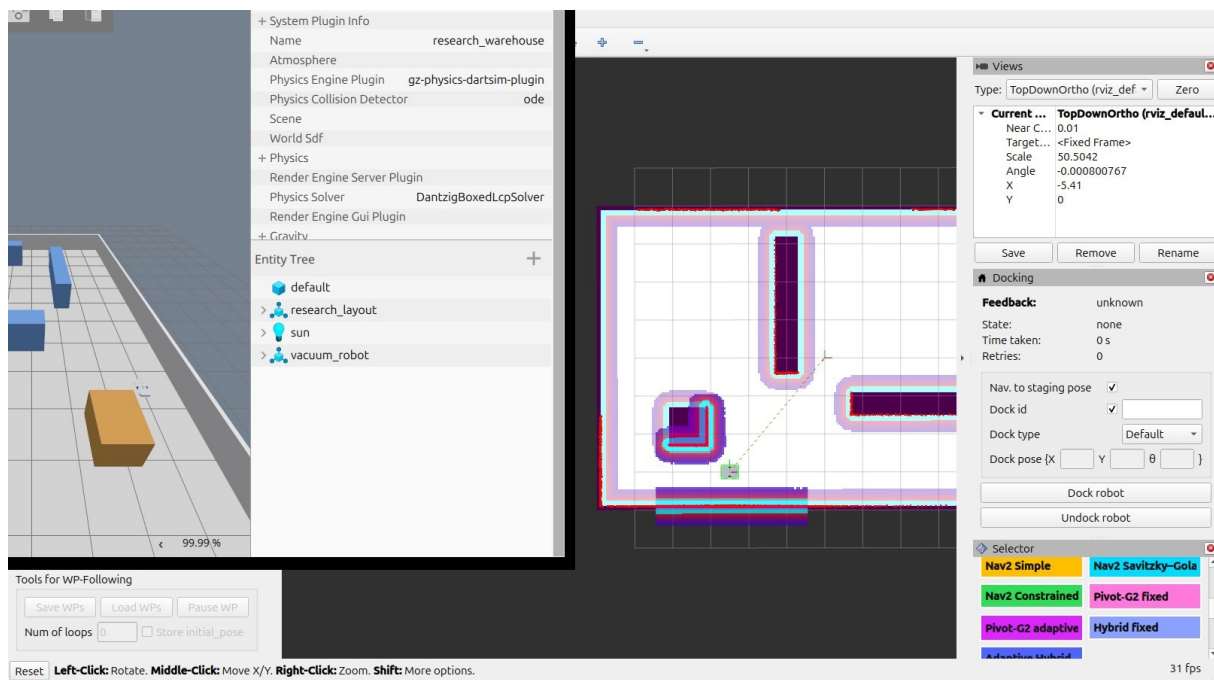
8.3 RViz2 và Gazebo không phải cùng một thứ

RViz2 chỉ hiển thị message ROS như map, TF, footprint, path, laser và marker. Nó không tích phân lực, ma sát hay va chạm. Gazebo mới là nơi mô phỏng robot có khối lượng, bánh xe, contact, sensor và physics. Vì vậy một path nhìn đẹp trong RViz2 chưa chứng minh robot thực hiện tốt. Dữ liệu cuối cùng phải lấy từ ground truth Gazebo hoặc robot thật.

Dự án có panel riêng để đổi môi trường, chọn planner, bật tắt từng smoother và chọn phương pháp thực thi. Environment manager chịu trách nhiệm dừng stack cũ, đổi đúng cặp world/map rồi khởi động stack mới để tránh RViz hiển thị một map nhưng Gazebo chạy world khác.



Giao diện chạy thực trong dự án: Gazebo mô phỏng vật lý, RViz2 hiển thị map, path và panel lựa chọn.

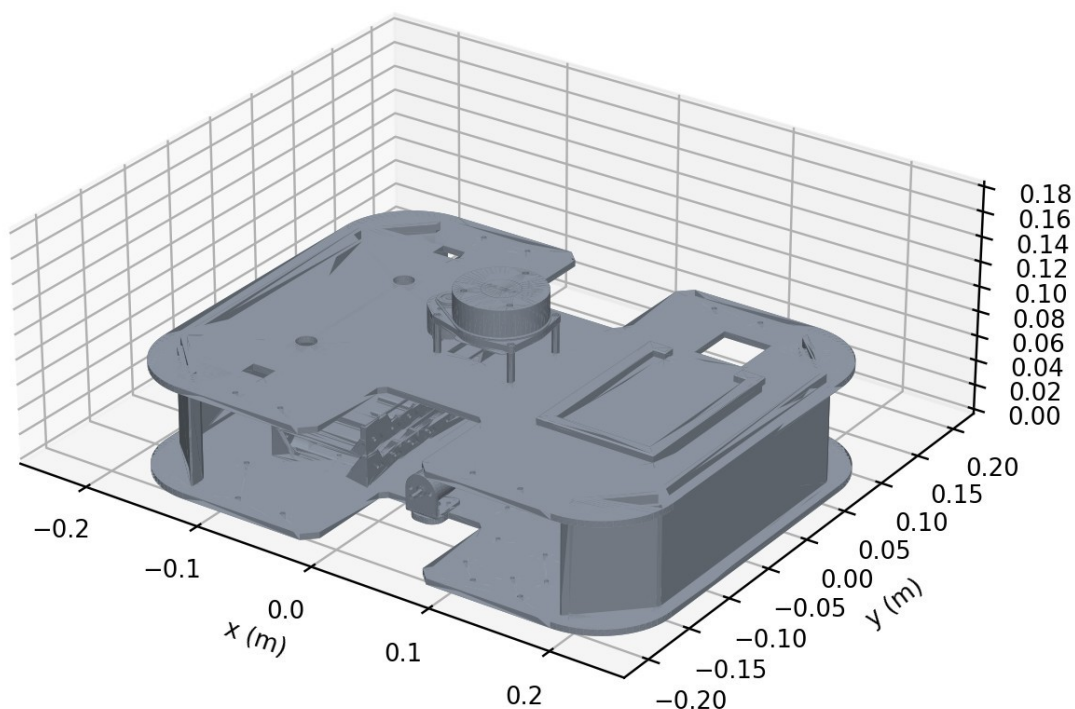


Panel RViz2 tách riêng planner, môi trường và từng phương pháp làm mượt để quan sát và so sánh.

IX, Mô hình robot, cảm biến và các giới hạn đang dùng

Mô hình CAD 3D dùng trực tiếp trong Gazebo
440 × 340 mm, bánh Ø85 mm, khối lượng 5,0 kg

- RPLIDAR A1M8
- BNO055



Mô hình CAD 3D của robot được nạp trực tiếp vào Gazebo.

9.1 Hình học, khối lượng và footprint

Chassis có envelope 440 × 340 mm, khối lượng 4,6 kg. Hai bánh mỗi bánh 0,2 kg, đường kính 85 mm, nên tổng khối lượng danh nghĩa khoảng 5,0 kg. Footprint Nav2 là hình chữ nhật có bốn đỉnh (0,22; 0,17), (0,22; -0,17), (-0,22; -0,17), (-0,22; 0,17).

Khoảng cách hai tâm vệt lăn vật lý theo CAD là 0,2548 m và được dùng trong động học, giới hạn vận tốc bánh và profile vận tốc. Plugin DiffDrive trong Gazebo dùng khoảng cách hiệu dụng 0,2834 m, tương ứng multiplier 1,1122448980, để odometry khớp ground truth của contact model. Giá trị 0,2834 m là kết quả làm tròn của fit bình phương tối thiểu có trọng số 0,283385 m từ hai trial độc lập ngày 26/07/2026. Hai giá trị không được trộn lẫn: 0,2548 m mô tả hình học thật, còn 0,2834 m chỉ hiệu chỉnh quan hệ bánh–thân xe trong mô phỏng.

9.2 Đọc đúng thông số của động cơ GA25

Mỗi bên xe dùng một động cơ giảm tốc GA25 có encoder, điện áp định mức 12 V DC và tỷ số truyền 45:1. Cụm từ 130 rpm trong tên thường chỉ tốc độ trục ra ở chế độ không tải danh nghĩa, không có nghĩa bánh xe luôn chạy 130 vòng/phút khi mang toàn bộ khối lượng robot. Để đưa thông số thương mại vào URDF, SDF và controller mà không làm sai bản chất, cần tách rõ điện áp, dòng, tốc độ, tỷ số truyền và mô-men.

Thông số GA25	Giá trị	Ý nghĩa và lý do sử dụng
Số lượng và loại	2 × GA25 encoder 130 rpm	Hai bánh chủ động độc lập
Điện áp định mức	12 V DC	Điện áp rail motor sau bộ hạ áp
Chiều quay	CW/CCW	Driver phải điều khiển được hai chiều
Tỷ số truyền	45:1	45 vòng armature xấp xỉ 1 vòng trục ra
Tốc độ armature	6000 rpm	Tốc độ phần motor trước hộp số
Không tải	130 ±10% rpm; 60 mA/motor	Giới hạn khi gần như không mang tải
Tải định mức	100 ±10% rpm; 300 mA/motor	Điểm làm việc danh nghĩa
Mô-men định mức	1 kgf·cm = 0,0980665 N·m	Mức tải dùng liên tục theo dữ liệu cung cấp
Stall	1,3 A; 3,6 kgf·cm = 0,3530394 N·m	Giới hạn tuyệt đối, không được giữ lâu
Kích thước	dài 68 mm; đường kính 25 mm	Kích thước thân motor trong mô hình

Voltage, ký hiệu U và đơn vị volt (V), là hiệu điện thế cấp cho motor. Rated voltage nghĩa là điện áp định mức mà nhà sản xuất dùng để công bố các điểm làm việc. Current, ký hiệu I và đơn vị ampere (A), là dòng điện motor lấy từ nguồn. No-load current là dòng khi gần như không có tải cơ; rated-load current là dòng tại tải danh nghĩa; stall current là dòng khi rotor bị khóa. Ba giá trị không được thay thế cho nhau trong thiết kế driver hoặc dây nguồn.

Speed có đơn vị rpm, viết đầy đủ là revolutions per minute, tức số vòng trong một phút. ROS và Gazebo thường dùng radian trên giây nên phải đổi đơn vị. Một vòng bằng 2π radian và một phút bằng 60 giây.

$$\omega = 2\pi n/60$$

Trong đó: n là tốc độ trục ra theo rpm; ω là vận tốc góc theo rad/s; π xấp xỉ 3,14159265.

Ý nghĩa và lý do sử dụng: 130 rpm tương ứng 13,613568 rad/s; 100 rpm tương ứng 10,471976 rad/s. Giá trị 13,613568 rad/s được giữ trong URDF/SDF như giới hạn vận tốc tuyệt đối của joint, còn 100 rpm phản ánh vùng tải định mức thực tế hơn.

Torque là mô-men xoắn, đo khả năng motor tạo lực quay. Dữ liệu đầu vào dùng kgf·cm, trong khi URDF dùng N·m. Một kgf là lực do khối lượng 1 kg chịu gia tốc trọng trường tiêu chuẩn 9,80665 m/s²; cánh tay đòn 1 cm bằng 0,01 m.

$$\tau[\text{N}\cdot\text{m}] = \tau[\text{kgf}\cdot\text{cm}] \times 9,80665 \times 0,01 = \tau[\text{kgf}\cdot\text{cm}] \times 0,0980665$$

Trong đó: τ là mô-men xoắn. Do đó 1 kgf·cm bằng 0,0980665 N·m và 3,6 kgf·cm bằng 0,3530394 N·m.

Ý nghĩa và lý do sử dụng: mô-men 0,0980665 N·m là điểm định mức; 0,3530394 N·m chỉ là stall torque. Không được dùng stall torque như mô-men làm việc liên tục, vì tại đó mỗi motor hút 1,3 A nhưng trục không quay, công suất điện chủ yếu biến thành nhiệt trong cuộn dây.

9.3 Từ tốc độ trục motor đến tốc độ tuyến tính của robot

Bánh xe có đường kính 0,085 m nên bán kính $r = 0,0425$ m. Nếu bỏ qua biến dạng lốp và trượt tiếp xúc, vận tốc tuyến tính tại vành bánh bằng tích của bán kính và vận tốc góc trục ra.

$$v_{\text{wheel}} = r_{\text{wheel}} \cdot \omega_{\text{output}} = r_{\text{wheel}} \cdot 2\pi n_{\text{output}} / 60$$

Trong đó: v_{wheel} là vận tốc tiếp tuyến bánh; $r_{\text{wheel}} = 0,0425$ m; ω_{output} là vận tốc góc trục ra; n_{output} là tốc độ rpm sau hộp số.

Với 130 rpm, tốc độ tuyến tính lý thuyết không tải bằng 0,578577 m/s. Với 100 rpm, tốc độ lý thuyết tại tải định mức bằng 0,445059 m/s. Controller chỉ cho thân xe tối đa 0,30 m/s và profile giới hạn tốc độ bánh ở 0,36 m/s. Như vậy lệnh điều khiển nằm dưới tốc độ tải định mức lý thuyết, tạo khoảng dự trữ cho quay vì sai, sai số điện áp, tải hàng và tổn hao cơ khí.

Khi robot quay, một bánh có thể nhanh hơn vận tốc tâm thân xe. Với vận tốc thân v và độ cong κ , vận tốc hai bánh được xấp xỉ bởi $v_R = v(1 + L\kappa/2)$ và $v_L = v(1 - L\kappa/2)$. Vì vậy kiểm tra $v \leq 0,30$ m/s chưa đủ; profile phải tiếp tục kiểm tra $\max(|v_R|, |v_L|) \leq 0,36$ m/s ở từng mẫu.

9.4 Cách biểu diễn GA25 trong URDF, SDF và transmission

URDF mô tả cấu trúc link–joint, hình học, khối lượng và giới hạn joint để ROS biết robot gồm những phần nào. SDF mô tả mô hình được Gazebo tích phân trong physics. Hai file cùng chứa thân motor GA25 dài 68 mm, đường kính 25 mm dưới dạng visual. Các motor visual không có collision và inertial riêng vì khối lượng motor đã nằm trong tham số chassis; nếu cộng thêm lần nữa sẽ làm sai tổng khối lượng và mô-men quán tính.

Hai transmission loại SimpleTransmission dùng mechanical reduction 45:1 để ghi rõ quan hệ armature–trục bánh. Joint bánh giữ giới hạn vận tốc 13,613568 rad/s và effort 0,3530394 N·m như hard absolute limits. Đây là giới hạn để chặn cấu hình phi vật lý, không phải lệnh mặc định mà controller cố đạt.

SDF DiffDrive nhận lệnh vận tốc thân xe và tạo chuyển động bánh theo contact model. Vì plugin mô phỏng không phải mô hình điện chi tiết của GA25, các số dòng 60 mA, 300 mA và 1,3 A hiện được lưu trong hardware profile chứ chưa được dùng để tính sụt áp hoặc nhiệt. Điều này phải được ghi rõ để người đọc không nhầm việc đã thêm thông số vào URDF với việc đã mô phỏng đầy đủ mạch điện và động cơ DC.

9.5 Encoder: những gì đã biết và những gì tuyệt đối không được đoán

Tên sản phẩm có chữ encoder nhưng dữ liệu được cung cấp chưa nêu encoder nằm ở trục armature hay trục ra, số xung mỗi vòng là bao nhiêu, và driver đếm cạnh x1, x2 hay x4. Vì vậy

các trường `pulses_per_revolution`, `quadrature_decode`, `encoder_ticks_per_rev`, `radians_per_tick` và `metres_per_tick` đang được để null trong `real_robot_profile.yaml`. Giá trị null ở đây là một ràng buộc an toàn dữ liệu, không phải thiếu sót cần lấp bằng một con số lấy từ motor GA25 khác.

$$N_{\text{tick,out}} = \text{PPR} \cdot q \cdot G \quad (\text{nếu encoder đặt trước hộp số})$$

$$\Delta\phi_{\text{wheel}} = 2\pi/N_{\text{tick,out}}, \quad \Delta s_{\text{wheel}} = r_{\text{wheel}} \cdot \Delta\phi_{\text{wheel}}$$

Trong đó: PPR là số pulse mỗi vòng theo cách nhà sản xuất định nghĩa; q là hệ số giải mã quadrature, có thể là 1, 2 hoặc 4; G = 45 là tỷ số truyền; $N_{\text{tick,out}}$ là số count cho một vòng trục ra; $\Delta\phi_{\text{wheel}}$ là góc bánh trên mỗi count; Δs_{wheel} là quãng đường lý thuyết trên mỗi count.

Ý nghĩa và lý do sử dụng: chỉ được nhân G khi encoder nằm ở phía armature. Nếu encoder đã nằm ở trục ra thì nhân thêm 45 sẽ làm sai odometry đúng 45 lần. Trước khi viết driver robot thật, cần đánh dấu một cạnh bánh, quay đúng một vòng trục ra, đọc count thô trên cả hai kênh và ghi rõ chế độ giải mã. Sau đó mới tính `metres_per_tick` và hiệu chuẩn bán kính bánh bằng quãng đường đo thực.

9.6 Bộ nguồn 16 cell 18650 mắc 4S4P

Ký hiệu 4S4P có nghĩa bốn cell nối tiếp trong mỗi nhánh và bốn nhánh như vậy nối song song. Nối tiếp làm tăng điện áp nhưng không tăng dung lượng Ah của một nhánh; song song làm tăng dung lượng và khả năng cấp dòng nhưng không tăng điện áp. Tổng số cell bằng tích của số cell nối tiếp và song song.

Đại lượng nguồn	Phép tính	Ý nghĩa và lý do sử dụng
Tổng số cell	$N = S \cdot P = 4 \cdot 4 = 16$	Đúng cấu hình 4S4P
Dung lượng một cell	2600 mAh = 2,6 Ah	Dung lượng điện tích danh nghĩa
Dòng xả ghi trên cell	$5C = 13 \text{ A}$	$13 \text{ A} = 5 \times 2,6 \text{ Ah}$
Điện áp pack danh nghĩa	$4 \times 3,7 = 14,8 \text{ V}$	Giả định Li-ion chuẩn, cần đối chiếu cell thật
Điện áp pack đầy	$4 \times 4,2 = 16,8 \text{ V}$	Không được cấp thẳng vào GA25 12 V
Dung lượng pack	$4 \times 2,6 = 10,4 \text{ Ah}$	Các nhánh song song cộng dung lượng
Năng lượng danh nghĩa	$14,8 \times 10,4 = 153,92 \text{ Wh}$	Ước lượng năng lượng lý thuyết
Dòng xả lý thuyết của cell-array	$4 \times 13 = 52 \text{ A}$	Không phải dòng an toàn mặc định của toàn xe
Rail motor	12 V đã điều áp	Bắt buộc dùng buck phù hợp trước driver

$$N_{\text{cell}} = S \cdot P = 4 \cdot 4 = 16$$

$$U_{\text{pack}} = S \cdot U_{\text{cell}}, \quad Q_{\text{pack}} = P \cdot Q_{\text{cell}}$$

$$E_{\text{nom}} \approx U_{\text{nom}} \cdot Q_{\text{pack}} = 14,8 \cdot 10,4 = 153,92 \text{ Wh}$$

Trong đó: S = 4 là số cell nối tiếp; P = 4 là số nhánh song song; U_{cell} là điện áp một cell; Q_{cell} là dung lượng một cell; U_{pack} và Q_{pack} là điện áp, dung lượng của pack; E_{nom} là năng lượng danh nghĩa.

Các giá trị 3,7 V danh nghĩa và 4,2 V khi đầy là giả định hóa học Li-ion tiêu chuẩn, không phải thông số đã được xác minh từ mã cell cụ thể. Với giả định đó, pack là 14,8 V danh nghĩa và 16,8 V khi đầy. Vì GA25 định mức 12 V, pack 4S đầy tuyệt đối không được cấp trực tiếp vào motor. Cần một buck converter tạo rail 12 V ổn định trước motor driver.

$$I_{\text{cell}} = C \cdot Q_{\text{cell}} = 5 \cdot 2,6 = 13 \text{ A}, \quad I_{\text{array,ideal}} = P \cdot I_{\text{cell}} = 52 \text{ A}$$

Trong đó: $C = 5$ là C-rate; $Q_{\text{cell}} = 2,6 \text{ Ah}$; $I_{\text{cell}} = 13 \text{ A}$ là dòng xả được ghi cho một cell; $I_{\text{array,ideal}} = 52 \text{ A}$ là tổng lý thuyết của bốn nhánh song song.

Ý nghĩa và lý do sử dụng: 52 A không phải dòng mà xe sẽ luôn lấy và cũng không tự động là dòng an toàn của hệ thống. Giới hạn an toàn thực bằng phần tử có rating thấp nhất trong chuỗi cell, mỗi hàn, BMS 4S, cầu chì, dây, connector, buck, PCB, motor driver và khả năng tản nhiệt. Model BMS, fuse, buck và driver hiện chưa được cung cấp nên các rating tương ứng phải tiếp tục để null.

Hai motor lấy tổng khoảng 0,12 A ở không tải, 0,60 A tại tải định mức và có thể đạt 2,60 A nếu cả hai cùng stall. Stall vẫn phải bị ngăn bằng giới hạn dòng, timeout lệnh 0,25 s, phát hiện bánh không quay và bảo vệ nhiệt; việc pack có thể cấp dòng lớn hơn không làm stall trở thành một chế độ vận hành hợp lệ.

9.7 Cảm biến

Cảm biến	Cấu hình mô phỏng	Topic	Vai trò
RPLIDAR A1M8	1440 tia, 5,5 Hz, 0,15–12 m, nhiễu 0,01 m	/scan	AMCL và local costmap
BNO055 IMU	100 Hz, nhiễu gyro/accel	/imu/data	Quan sát quay và tích hợp tương lai
Wheel odometry	30 Hz từ DiffDrive	/odom	Cập nhật odom → base_link
Gazebo ground truth	30 Hz, độc lập odometry	/ground_truth/odom	Chấm sai số vật lý

9.8 Bảng tham số chính

Tham số	Giá trị	Vai trò
Footprint	$0,44 \times 0,34 \text{ m}$	Bao toàn thân robot trong costmap
Khoảng vệt lăn L	0,2548 m	Động học và giới hạn bánh
Đường kính bánh	0,085 m	Mô hình Gazebo
v_{max}	0,30 m/s	Vận tốc tuyến tính tối đa
ω_{max}	0,80 rad/s	Vận tốc góc tối đa
$v_{w,\text{max}}$	0,36 m/s	Vận tốc dài tối đa mỗi bánh
$a_{y,\text{max}}$	0,18 m/s ²	Gia tốc ngang trong cong

Tham số	Giá trị	Vai trò
a tăng	0,35 m/s ²	Gia tốc tiến tối đa
a giảm	0,45 m/s ²	Độ lớn giảm tốc tối đa
$\alpha_{\max}$	1,20 rad/s ²	Gia tốc góc tối đa
jmax	0,90 m/s ³	Jerk tuyến tính tối đa
Sample transition	0,02 m	Kiểm tra hình học Bézier
Output spacing	0,05 m	Resample path đầu ra
R thích nghi	0,10...1,50 m	Miền bán kính thiết kế
Ngân sách mỗi góc	6 mẫu đầu, tối đa 20	Coarse-to-fine
Candidate giữ lại	5 G ² /góc + Pivot	Trạng thái cho DP
Footprint cost tối đa	200	Gate center cost trước footprint
Khoảng vật lăn hiệu dụng Gazebo	0,2834 m	Hiệu chuẩn contact/odometry, không thay CAD
GA25	2 × 12 V, 45:1, 130 rpm	Nguồn động lực hai bánh
Giới hạn joint tuyệt đối	13,613568 rad/s; 0,3530394 N·m	No-load speed và stall torque
Tốc độ tải định mức lý thuyết	0,445059 m/s	Mốc vật lý cao hơn controller 0,30 m/s
Nguồn	16 × 18650, 4S4P, 10,4 Ah	Pack 14,8 V danh nghĩa; 16,8 V đầy
Rail motor	12 V qua buck	Không cấp thẳng 4S đầy vào GA25
Encoder PPR	Chưa biết, để null	Không suy đoán sai odometry robot thật

X, Map, costmap và bầy môi trường kiểm chứng

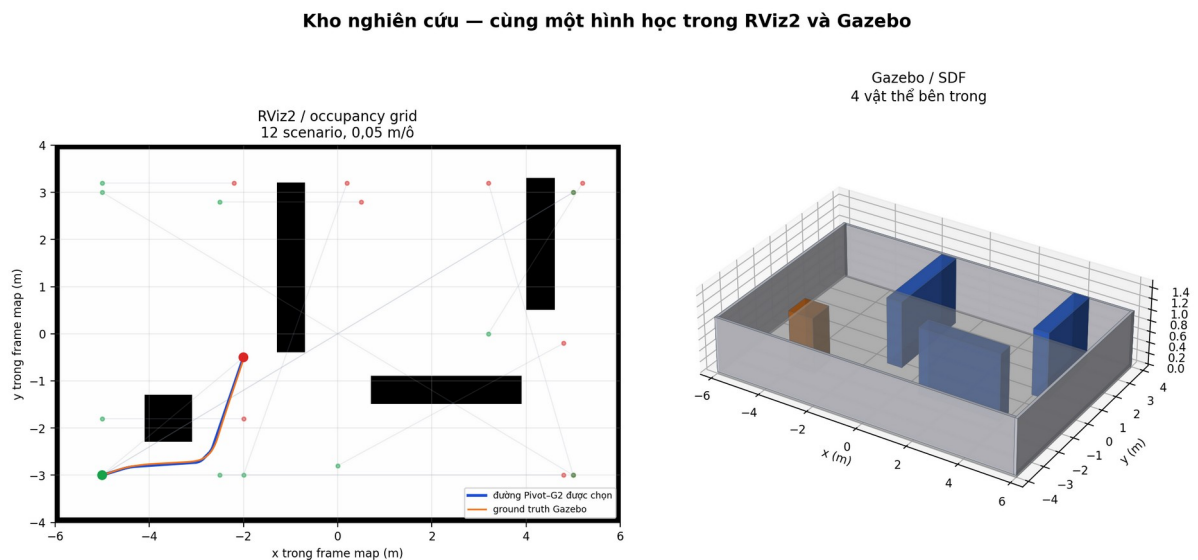
10.1 Một môi trường được tạo từ những file nào?

Mỗi môi trường có ba phần đồng bộ. File .pgm là ảnh occupancy mà map server và RViz2 đọc. File .yaml chứa resolution, origin và ngưỡng occupied/free. File .sdf mô tả sàn, tường, kệ, vật cản, ánh sáng và physics cho Gazebo. PGM và SDF được sinh từ cùng một hình học để vật cản nhìn thấy trong RViz2 trùng với vật cản vật lý trong Gazebo.

Cả bầy map có kích thước 12×8 m, resolution 0,05 m/ô và origin (-6; -4; 0). occupied_thresh bằng 0,65, free_thresh bằng 0,25. Inflation layer lan chi phí với radius 0,45 m. Center cost hữu ích để đánh giá proximity nhưng không thay collision footprint.

10.2 Kho nghiên cứu - research_warehouse

Môi trường tổng hợp gồm ba kệ khác hướng và một thùng hàng. Nó có góc vuông, đường chéo, lối hẹp cục bộ và đoạn thẳng dài nên được dùng làm map phát triển và kiểm tra hồi quy chính.



Kho nghiên cứu: occupancy grid RViz2, hình học SDF Gazebo, selected path và ground truth lấy từ trace cuối ngày 26/07/2026. Các điểm start-goal mờ là toàn bộ scenario có trong map.

Scenario đại diện lower_left_diagonal chạy với ThetaStar, Pivot-G² thích nghi và profile vận tốc hiện tại. Thời gian hoàn thành là 35,313 s, ground-truth RMSE là 2,148 cm, sai số cực đại là 3,519 cm, exit RMSE là 2,375 cm và clearance footprint kế hoạch là 24,749 cm. Đây là trace cuối ngày 26/07/2026, đã đạt action success, đạt ground-truth goal và dừng vật lý; số liệu không chỉ phản ánh smoother mà còn chứa ảnh hưởng của định vị, controller và physics Gazebo.

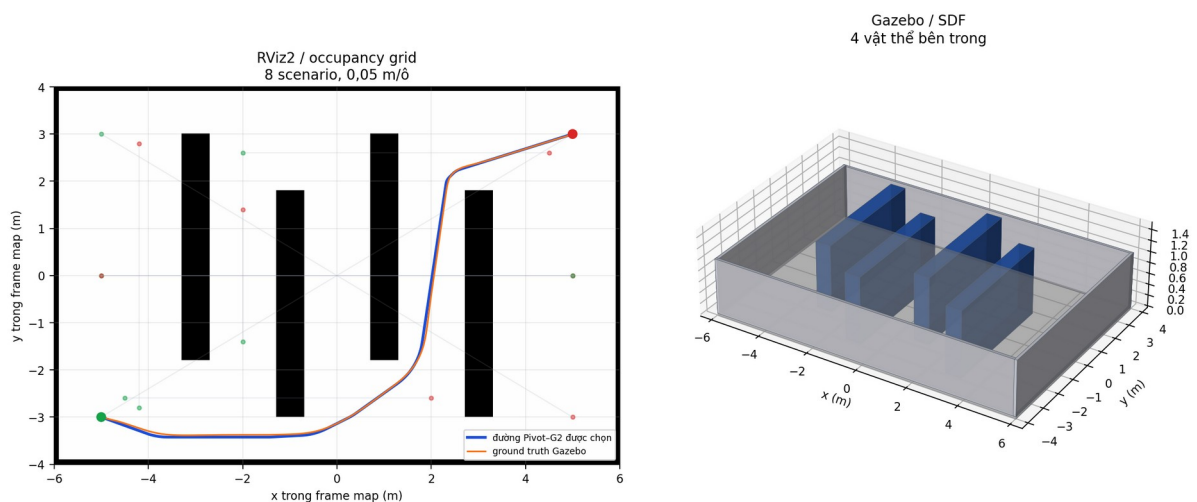
Scenario	Start (m)	Goal (m)
straight_lower_aisle	(-2,50; -3,00)	(4,80; -3,00)

Scenario	Start (m)	Goal (m)
crate_detour	(-5,00; -1,80)	(-2,00; -1,80)
vertical_rack_detour	(-2,50; 2,80)	(0,50; 2,80)
horizontal_rack_detour	(0,00; -2,80)	(4,80; -0,20)
right_rack_detour	(3,20; 0,00)	(5,20; 3,20)
long_southwest_to_northeast	(-5,00; -3,00)	(5,00; 3,00)
long_northeast_to_southwest	(5,00; 3,00)	(-5,00; -3,00)
crossing_northwest_to_southeast	(-5,00; 3,00)	(5,00; -3,00)
central_south_to_north	(-2,00; -3,00)	(0,20; 3,20)
right_south_to_north	(5,00; -3,00)	(3,20; 3,20)
upper_open_short	(-5,00; 3,20)	(-2,20; 3,20)
lower_left_diagonal	(-5,00; -3,00)	(-2,00; -0,50)

10.3 Lối đi hẹp - narrow_aisles

Bốn dãy kệ dọc đặt lệch nhau tạo hành lang ngoằn ngoèo. Map này nhấn mạnh clearance của footprint, các góc liên tiếp và lối projection chọn nhằm nhánh khi hai đoạn nằm gần nhau.

Lối đi hẹp — cùng một hình học trong RViz2 và Gazebo



Lối đi hẹp: occupancy grid RViz2, hình học SDF Gazebo, selected path và ground truth lấy từ trace cuối ngày 26/07/2026. Các điểm start-goal mờ là toàn bộ scenario có trong map.

Scenario đại diện southwest_northeast_weave chạy với ThetaStar, Pivot-G² thích nghi và profile vận tốc hiện tại. Thời gian hoàn thành là 67,134 s, ground-truth RMSE là 3,346 cm, sai số cực đại là 6,565 cm, exit RMSE là 3,154 cm và clearance footprint kế hoạch là 14,492 cm. Đây là trace cuối ngày 26/07/2026, đã đạt action success, đạt ground-truth goal và dừng vật lý;

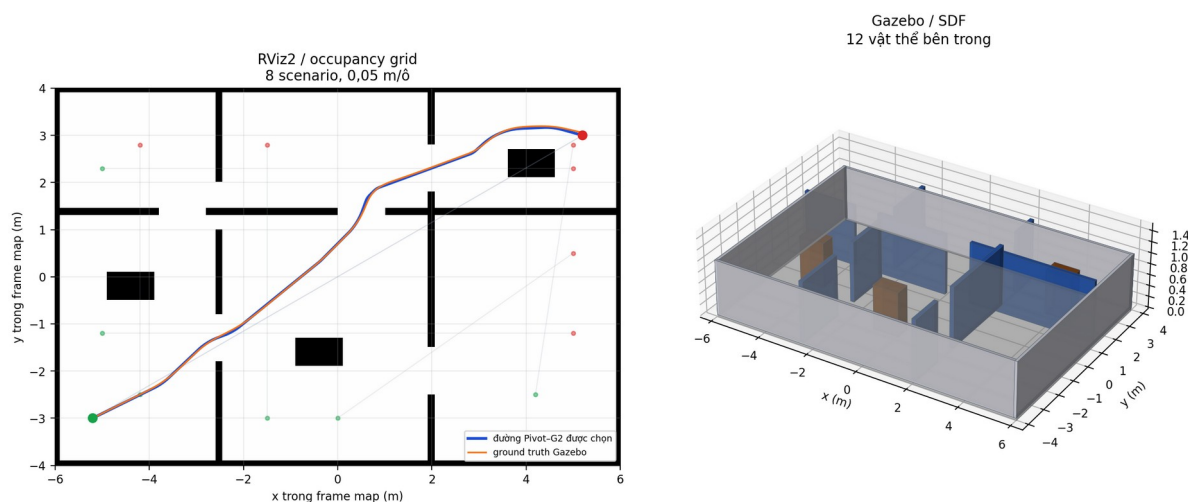
số liệu không chỉ phản ánh smoother mà còn chứa ảnh hưởng của định vị, controller và physics Gazebo.

Scenario	Start (m)	Goal (m)
serpentine_west_east	(-5,00; 0,00)	(5,00; 0,00)
serpentine_east_west	(5,00; 0,00)	(-5,00; 0,00)
left_vertical_aisle	(-4,20; -2,80)	(-4,20; 2,80)
inner_vertical_aisle	(-2,00; -1,40)	(-2,00; 1,40)
bottom_alternating_cross	(-4,50; -2,60)	(2,00; -2,60)
top_alternating_cross	(-2,00; 2,60)	(4,50; 2,60)
southwest_northeast_weave	(-5,00; -3,00)	(5,00; 3,00)
northwest_southeast_weave	(-5,00; 3,00)	(5,00; -3,00)

10.4 Mê cung văn phòng - office_maze

Các vách ngăn đứt đoạn, cửa lệch nhau và ba bàn làm việc tạo nhiều đường chữ L, U và ngõ cụt. Đây là bài thử cho planner và cho DP khi hai góc liên tiếp dùng chung một đoạn ngắn.

Mê cung văn phòng — cùng một hình học trong RViz2 và Gazebo



Mê cung văn phòng: occupancy grid RViz2, hình học SDF Gazebo, selected path và ground truth lấy từ trace cuối ngày 26/07/2026. Các điểm start-goal mờ là toàn bộ scenario có trong map.

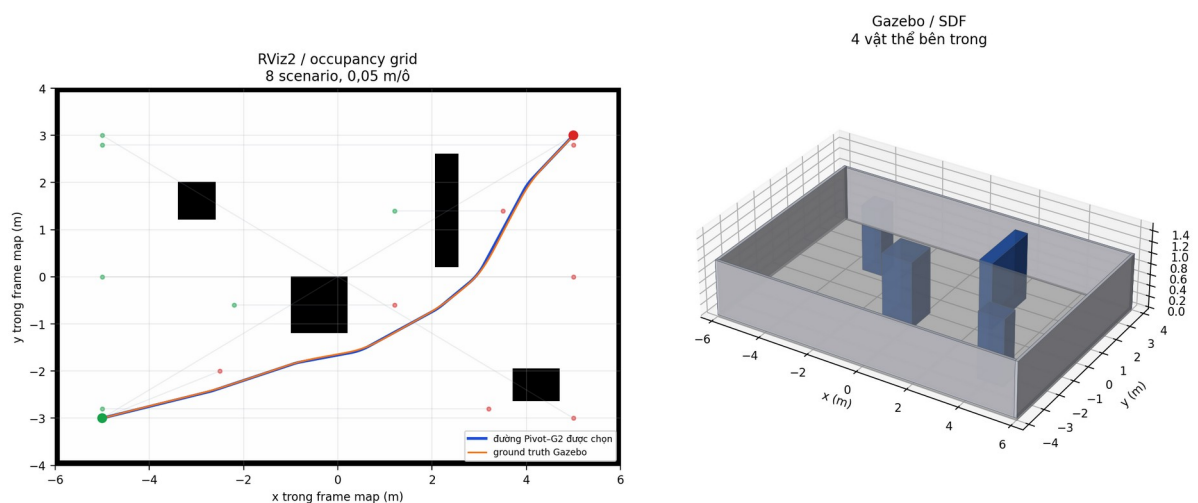
Scenario đại diện office_long_diagonal chạy với ThetaStar, Pivot-G² thích nghi và profile vận tốc hiện tại. Thời gian hoàn thành là 75,303 s, ground-truth RMSE là 1,809 cm, sai số cực đại là 4,799 cm, exit RMSE là 3,145 cm và clearance footprint kế hoạch là 18,825 cm. Đây là trace cuối ngày 26/07/2026, đã đạt action success, đạt ground-truth goal và dừng vật lý; số liệu không chỉ phản ánh smoother mà còn chứa ảnh hưởng của định vị, controller và physics Gazebo.

Scenario	Start (m)	Goal (m)
office_long_diagonal	(-5,20; -3,00)	(5,20; 3,00)
office_reverse_diagonal	(5,20; 3,00)	(-5,20; -3,00)
west_room_to_room	(-4,20; -2,50)	(-4,20; 2,80)
east_room_to_room	(4,20; -2,50)	(5,00; 2,80)
lower_cross_offices	(-5,00; -1,20)	(5,00; -1,20)
upper_cross_offices	(-5,00; 2,30)	(5,00; 2,30)
central_u_turn	(-1,50; -3,00)	(-1,50; 2,80)
east_partition_detour	(0,00; -3,00)	(5,00; 0,50)

10.5 Không gian mở - open_arena

Không gian thưa với cột, khối vuông và vách chắn rời rạc. Map này tách ảnh hưởng của bản thân thuật toán khỏi ảnh hưởng hành lang hẹp, phù hợp so sánh chiều dài, thời gian và đường chéo.

Không gian mở — cùng một hình học trong RViz2 và Gazebo



Không gian mở: occupancy grid RViz2, hình học SDF Gazebo, selected path và ground truth lấy từ trace cuối ngày 26/07/2026. Các điểm start-goal mở là toàn bộ scenario có trong map.

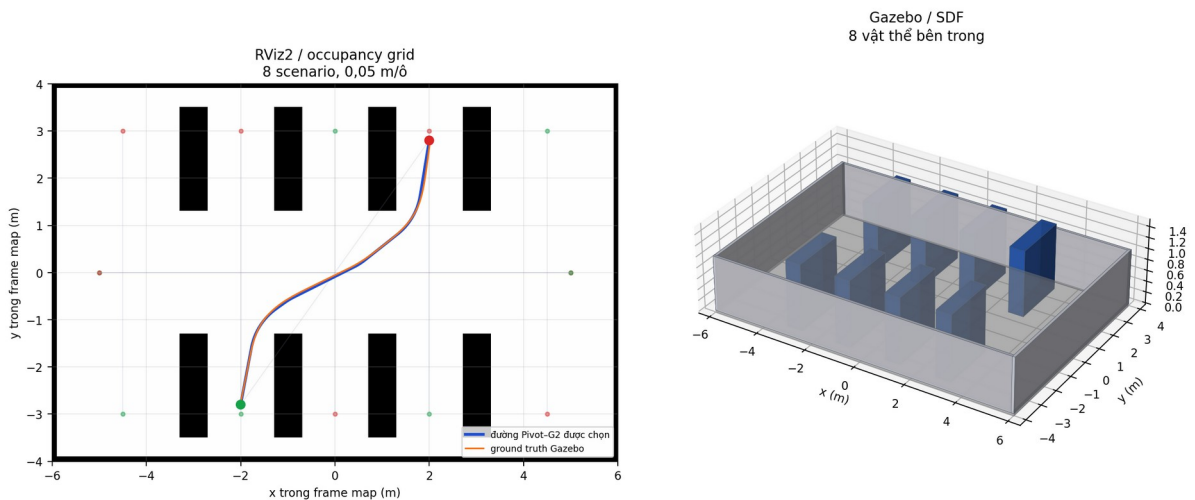
Scenario đại diện southwest_northeast chạy với ThetaStar, Pivot-G² thích nghi và profile vận tốc hiện tại. Thời gian hoàn thành là 53,052 s, ground-truth RMSE là 1,570 cm, sai số cực đại là 3,586 cm, exit RMSE là 1,692 cm và clearance footprint kế hoạch là 21,464 cm. Đây là trace cuối ngày 26/07/2026, đã đạt action success, đạt ground-truth goal và dừng vật lý; số liệu không chỉ phản ánh smoother mà còn chứa ảnh hưởng của định vị, controller và physics Gazebo.

Scenario	Start (m)	Goal (m)
west_east_center	(-5,00; 0,00)	(5,00; 0,00)
southwest_northeast	(-5,00; -3,00)	(5,00; 3,00)
northwest_southeast	(-5,00; 3,00)	(5,00; -3,00)
center_block_detour	(-2,20; -0,60)	(1,20; -0,60)
screen_detour	(1,20; 1,40)	(3,50; 1,40)
long_lower_lane	(-5,00; -2,80)	(3,20; -2,80)
long_upper_lane	(-5,00; 2,80)	(5,00; 2,80)
short_open_diagonal	(-5,00; -3,00)	(-2,50; -2,00)

10.6 Kho có lối giao cắt - warehouse_cross_aisles

Bốn dãy kệ được tách đôi ở giữa để tạo lối ngang rộng. Robot phải chuyển từ lối dọc sang lối ngang rồi trở lại lối dọc, nên đây là bài thử thách cho pha vào cong và thoát cong.

Kho có lối giao cắt — cùng một hình học trong RViz2 và Gazebo



Kho có lối giao cắt: occupancy grid RViz2, hình học SDF Gazebo, selected path và ground truth lấy từ trace cuối ngày 26/07/2026. Các điểm start-goal mờ là toàn bộ scenario có trong map.

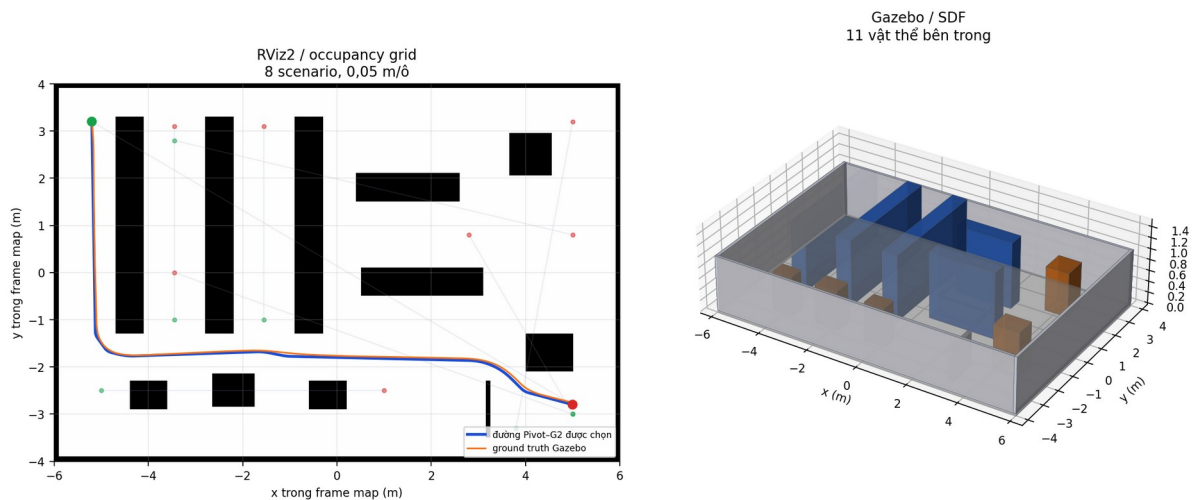
Scenario đại diện cross_aisle_transfer chạy với ThetaStar, Pivot-G² thích nghi và profile vận tốc hiện tại. Thời gian hoàn thành là 34,455 s, ground-truth RMSE là 2,229 cm, sai số cực đại là 4,152 cm, exit RMSE là 3,008 cm và clearance footprint kế hoạch là 14,492 cm. Đây là trace cuối ngày 26/07/2026, đã đạt action success, đạt ground-truth goal và dừng vật lý; số liệu không chỉ phản ánh smoother mà còn chứa ảnh hưởng của định vị, controller và physics Gazebo.

Scenario	Start (m)	Goal (m)
central_cross_west_east	(-5,00; 0,00)	(5,00; 0,00)
central_cross_east_west	(5,00; 0,00)	(-5,00; 0,00)
west_service_lane	(-4,50; -3,00)	(-4,50; 3,00)
picking_aisle_a	(-2,00; -3,00)	(-2,00; 3,00)
picking_aisle_b	(0,00; 3,00)	(0,00; -3,00)
picking_aisle_c	(2,00; -3,00)	(2,00; 3,00)
east_service_lane	(4,50; 3,00)	(4,50; -3,00)
cross_aisle_transfer	(-2,00; -2,80)	(2,00; 2,80)

10.7 Kho điều phối–xuất hàng - warehouse_dispatch

Kho hỗn hợp có vùng chứa hàng, pallet đầu vào, kệ staging, pallet đầu ra và vách dock. Đây là map khó nhất về mật độ vật cản và đã tạo ra ca phản ví dụ buộc Hybrid phải fallback.

Kho điều phối–xuất hàng — cùng một hình học trong RViz2 và Gazebo



Kho điều phối–xuất hàng: occupancy grid RViz2, hình học SDF Gazebo, selected path và ground truth lấy từ trace cuối ngày 26/07/2026. Các điểm start–goal mờ là toàn bộ scenario có trong map.

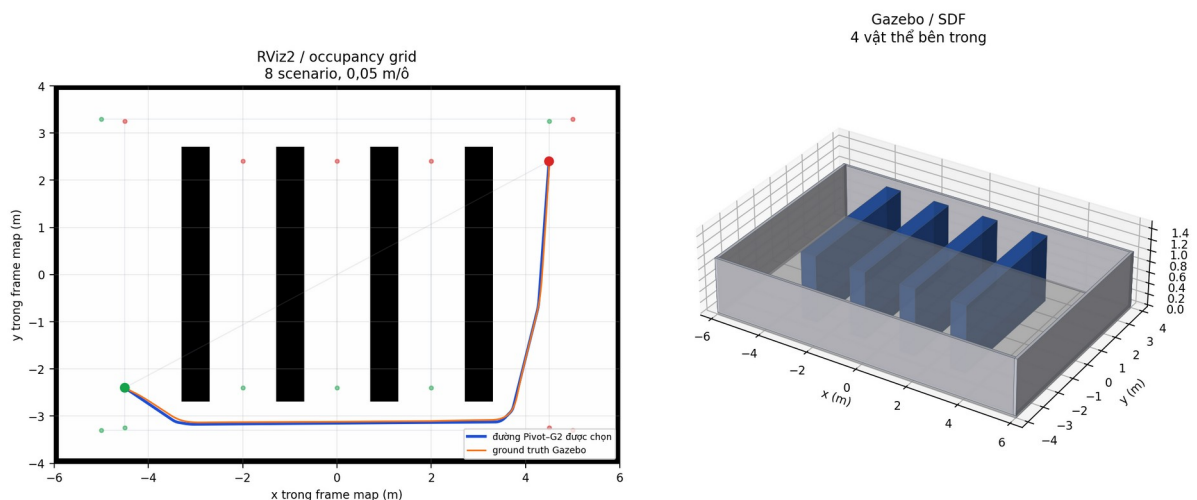
Scenario đại diện full_replenishment chạy với ThetaStar, Pivot-G² thích nghi và profile vận tốc hiện tại. Thời gian hoàn thành là 82,443 s, ground-truth RMSE là 3,453 cm, sai số cực đại là 7,371 cm, exit RMSE là 4,039 cm và clearance footprint kế hoạch là 10,607 cm. Đây là trace cuối ngày 26/07/2026, đã đạt action success, đạt ground-truth goal và dừng vật lý; số liệu không chỉ phản ánh smoother mà còn chứa ảnh hưởng của định vị, controller và physics Gazebo.

Scenario	Start (m)	Goal (m)
storage_aisle_a	(-3,45; -1,00)	(-3,45; 3,10)
storage_aisle_b	(-1,55; -1,00)	(-1,55; 3,10)
storage_to_dispatch	(-3,45; 2,80)	(5,00; 0,80)
receiving_to_storage	(5,00; -3,00)	(-3,45; 0,00)
inbound_pallet_slalom	(-5,00; -2,50)	(1,00; -2,50)
dispatch_lane_south_north	(3,80; -3,30)	(5,00; 3,20)
inbound_to_staging	(5,00; -3,00)	(2,80; 0,80)
full_replenishment	(-5,20; 3,20)	(5,00; -2,80)

10.8 Kho có lối đi dài - warehouse_long_aisles

Bốn kệ dài song song tạo các lối picking hẹp và hành lang chuyển tiếp ở hai đầu. Map kiểm tra quãng đường dài, tích lũy sai số định vị và việc tăng tốc lại sau nhiều đoạn cong.

Kho có lối đi dài — cùng một hình học trong RViz2 và Gazebo



Kho có lối đi dài: occupancy grid RViz2, hình học SDF Gazebo, selected path và ground truth lấy từ trace cuối ngày 26/07/2026. Các điểm start-goal mờ là toàn bộ scenario có trong map.

Scenario đại diện diagonal_replenishment chạy với ThetaStar, Pivot-G² thích nghi và profile vận tốc hiện tại. Thời gian hoàn thành là 76,953 s, ground-truth RMSE là 3,243 cm, sai số cực đại là 7,178 cm, exit RMSE là 2,623 cm và clearance footprint kế hoạch là 23,390 cm. Đây là trace cuối ngày 26/07/2026, đã đạt action success, đạt ground-truth goal và dừng vật lý; số liệu không chỉ phản ánh smoother mà còn chứa ảnh hưởng của định vị, controller và physics Gazebo.

Scenario	Start (m)	Goal (m)
west_service_lane	(-4,50; -3,25)	(-4,50; 3,25)
picking_aisle_a	(-2,00; -2,40)	(-2,00; 2,40)
picking_aisle_b	(0,00; -2,40)	(0,00; 2,40)
picking_aisle_c	(2,00; -2,40)	(2,00; 2,40)
east_service_lane	(4,50; 3,25)	(4,50; -3,25)
south_transfer_corridor	(-5,00; -3,30)	(5,00; -3,30)
north_transfer_corridor	(-5,00; 3,30)	(5,00; 3,30)
diagonal_replenishment	(-4,50; -2,40)	(4,50; 2,40)

XI, Benchmark đo gì và làm thế nào để tránh kết luận sai?

11.1 Hai tầng đánh giá

Tầng thứ nhất là benchmark hình học. Toàn bộ thiết kế gồm $7 \text{ map} \times 60 \text{ scenario} \times 5 \text{ planner} \times 8 \text{ phương pháp} \times 3 \text{ lần lặp}$, tạo 7.200 dòng dữ liệu. Mỗi nhóm tám phương pháp trong cùng điều kiện phải có cùng raw_path_sha256. Nhờ đó sự khác biệt được quy về smoother thay vì khác planner input.

Tầng thứ hai là chạy kín. Không thể thực hiện toàn bộ tích $7 \times 5 \times 8 \times 3$ với nhiều seed trong một lần phát triển vì chi phí mô phỏng rất lớn. Vì vậy ma trận chạy kín được phân tầng thành 42 trial, trong đó có 24 trial chính gồm tám method ở ba chế độ tốc độ và các trial bổ sung để kiểm tra planner, map và ca robust. Tổng ma trận đạt 41/42 thành công.

11.2 Hợp đồng dữ liệu của đợt kiểm chứng ngày 26/07/2026

Đợt rà soát mới không thay thế ma trận 7.200 dòng và 42 trial ngày 25/07. Nó là một lớp kiểm chứng hồi quy tập trung vào những thay đổi có nguy cơ làm xe sai hướng hoặc lệch khỏi đường sau cong. Bảy trace đại diện bao phủ đủ bảy environment; thêm hai cặp before–after ở lower_left_diagonal và right_rack_detour, cùng một cặp trước–sau luật phanh góc ở narrow_aisles.

Một trace chỉ được nhận khi action trả success, ground-truth pose nằm trong tolerance, ground-truth yaw đạt yêu cầu và robot đã dừng vật lý. selected_path_xy và ground_truth_state_trace phải tồn tại để kết quả có thể tính lại. Bốn độ dịch neo start–goal phải không vượt 0,08 m. Nominal jerk phải không vượt $0,9 \text{ m/s}^3$ sau khi loại đúng những mẫu safety override.

Báo cáo phân biệt nominal jerk và actual jerk. Nominal jerk đo những chu kỳ S-curve còn đầy đủ miền khả thi. Actual jerk cũng chứa các lần cap an toàn hoặc biên $v = 0$ buộc lệnh thay đổi nhanh hơn. Nếu chỉ báo actual jerk mà không kèm cờ override, người đọc có thể kết luận sai rằng thuật toán vi phạm giới hạn ở mọi thời điểm. Nếu chỉ báo nominal jerk, người đọc lại không nhìn thấy các can thiệp an toàn. Vì vậy hai đại lượng phải tồn tại song song.

Ground-truth tracking error đo khoảng cách từ pose vật lý Gazebo đến selected path. Estimated tracking error dùng pose trong TF mà controller nhìn thấy. Odometry error so odom với chuyển động vật lý sau căn hệ. Localization error so map pose ước lượng với ground truth. Việc tách bốn nguồn giúp xác định xe lệch vì path, vì controller, vì odometry hay vì AMCL thay vì gộp mọi lỗi vào một RMSE.

11.3 Định nghĩa metric

$$\text{RMSE} = \sqrt{[(1/M) \sum e_i^2]}$$

Trong đó: RMSE là căn trung bình bình phương sai số; M là số mẫu; e_i là sai số tại mẫu i.

Ý nghĩa và lý do sử dụng: Bình phương làm sai số lớn bị phạt mạnh hơn. Căn bậc hai đưa đơn vị trở lại mét hoặc centimet để dễ diễn giải.

Tracking RMSE ground truth là khoảng cách từ pose vật lý Gazebo đến segment path gần nhất. Estimated RMSE là sai số mà controller nhìn thấy trong hệ map/TF. Localization error là khoảng cách giữa pose ước lượng và ground truth đã căn chỉnh. Curve RMSE và exit RMSE chỉ đo tại vùng cong và vùng ngay sau cong. Clearance là khoảng hở nhỏ nhất của toàn footprint, không phải khoảng cách tâm. Success chỉ được tính khi action thành công, pose ground truth nằm trong tolerance, yaw đạt yêu cầu và robot đã dừng.

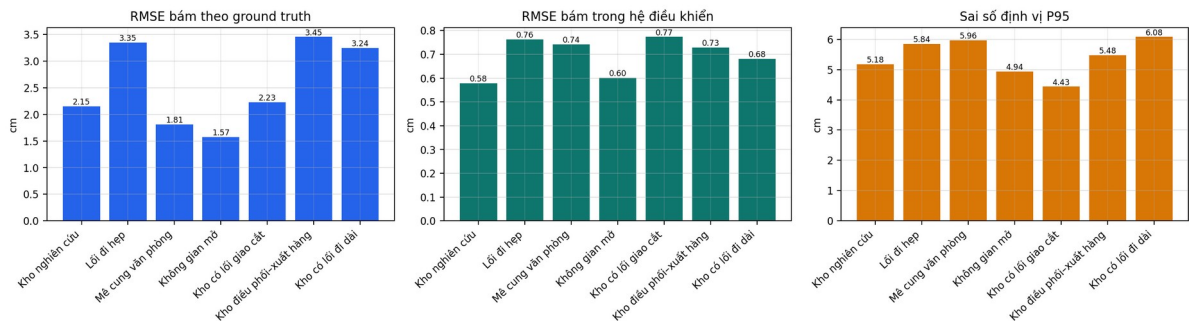
Việc tách các metric này giúp tránh một kết luận sai phổ biến: controller có thể báo sai số rất nhỏ vì nó đang bám theo pose ước lượng, trong khi robot vật lý đã lệch nhiều hơn do localization. Vì vậy báo cáo luôn đặt RMSE điều khiển cạnh ground-truth RMSE và localization P95.

XII, Kết quả hình học và chạy kín

12.1 Kết quả đại diện trên bảy môi trường

Map	Scenario	t (s)	GT RMSE (cm)	GT max (cm)	RMSE controller (cm)	Định vị P95 (cm)	Clearance (cm)
Kho nghiên cứu	lower_level_diagonal	35,313	2,148	3,519	0,579	5,176	24,749
Lối đi hẹp	southwest_northeast_weave	67,134	3,346	6,565	0,763	5,844	14,492
Mê cung văn phòng	office_long_diagonal	75,303	1,809	4,799	0,742	5,959	18,825
Không gian mở	southwest_northeast	53,052	1,570	3,586	0,600	4,935	21,464
Kho có lối giao cắt	cross_aisle_transfer	34,455	2,229	4,152	0,773	4,434	14,492
Kho điều phối-xuất hàng	full_replenishment	82,443	3,453	7,371	0,729	5,478	10,607
Kho có lối đi dài	diagonal_replenishment	76,953	3,243	7,178	0,682	6,078	23,390

Kiểm chứng chạy kín Pivot-G2 thích nghi trên 7 môi trường — trace cuối 26/07/2026



Ba nhóm sai số của bảy trace cuối ngày 26/07/2026. Chiều cao từng cột được sinh trực tiếp từ cùng JSON dùng để điền bảng ngay phía trên.

12.2 Bảng legacy đã công bố trong supplement ngày 25/07/2026

Metric	Trước	Sau	Cải thiện
Thời gian	53,094 s	38,265 s	27,9%
GT RMSE	1,883 cm	1,412 cm	25,0%
Sai số vị trí cuối	2,309 cm	1,071 cm	53,6%
Sai số yaw cuối	0,0138 rad	0,0061 rad	55,6%

Bảng phía trên được giữ nguyên theo docs/REV_ECIT_2026_ADAPTIVE_HYBRID_PIVOT_G2_SUPPLEMENT.html để lưu dấu vết kết quả đã công bố trước đợt audit hiện tại. Repository không còn giữ một cặp raw JSON độc lập đủ rõ để tái tính riêng bảng này, nên nó được ghi nhãn legacy và không được dùng làm bằng chứng cho phiên bản 26/07/2026. Về mặt lịch sử, bảng cho thấy projection, profile vận tốc và terminal servo đã ảnh hưởng trực tiếp đến thời gian, tracking error và sai số cuối như thế nào. Không dùng các số này để thay cho trace hiện tại; đợt kiểm chứng sau đây có dữ liệu, mã điều khiển và hiệu chuẩn Gazebo mới hơn.

12.3 Rà soát kín ngày 26/07/2026 sau khi sửa hướng và vận tốc

Đợt rà soát này được thực hiện từ dữ liệu Gazebo ground truth, estimated pose, odometry, lệnh vận tốc và telemetry controller. Mục tiêu không phải chọn riêng một metric đẹp nhất, mà kiểm tra xem các phần mới có cùng dẫn đến một chuyển động logic hay không: robot spawn đúng hướng có thể đi, selected path bắt đầu đúng vị trí, lệnh quay phanh trước target, xe không tăng tốc khi còn lệch heading, và sau cong có thể trở về đường thay vì trôi ra ngoài.

Scenario	Metric	Trước	Sau	Mức giảm
lower_left_diagonal	Thời gian (s)	35,610	35,313	0,8%
lower_left_diagonal	GT RMSE (cm)	2,112	2,148	-1,7%
lower_left_diagonal	Odom vị trí P95 (cm)	2,665	0,993	62,7%
lower_left_diagonal	Odom yaw P95 (rad)	0,012516	0,002107	83,2%
right_rack_detour	Thời gian (s)	44,718	36,870	17,5%
right_rack_detour	GT RMSE (cm)	2,185	1,588	27,3%
right_rack_detour	Odom vị trí P95 (cm)	3,850	1,792	53,5%
right_rack_detour	Odom yaw P95 (rad)	0,015159	0,002310	84,8%

Ở right_rack_detour, thời gian giảm 17,5%, GT RMSE giảm 27,3%, exit max giảm 34,4% và odometry yaw P95 giảm 84,8%. Đây là ca thể hiện rõ nhất lợi ích của việc đồng bộ hiệu chuẩn khoảng vệt lăn, neo path và profile quay. Ở lower_left_diagonal, odometry position P95 giảm 62,7%, odometry yaw P95 giảm 83,2% và estimated RMSE giảm 27,5%. GT RMSE của riêng lần chạy cuối tăng nhẹ khoảng 1,7%, từ 2,112 cm lên 2,148 cm; sai số cuối cũng biến động nhưng vẫn trong tolerance. Kết quả này được ghi thẳng thay vì chỉ giữ metric có lợi, vì hai lần mô phỏng có nhiều AMCL và không thể dùng một mẫu để tuyên bố mọi đại lượng đều cải thiện tuyệt đối.

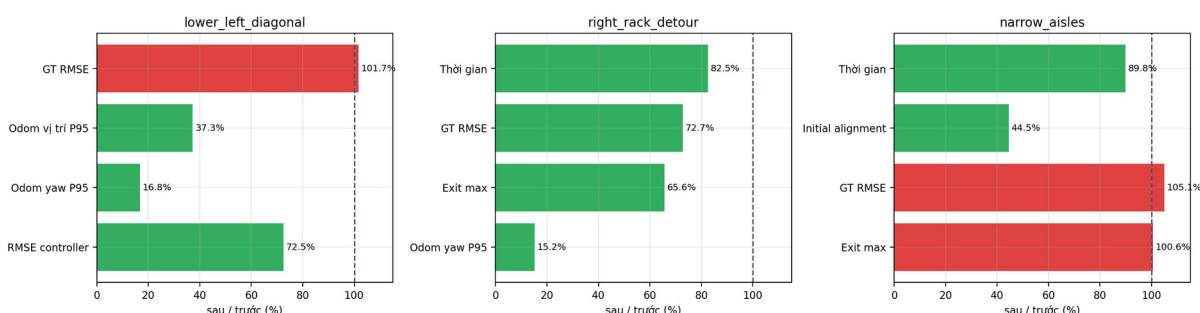
Metric narrow_aisles	Trước luật phanh	Sau luật phanh	Mức giảm
Thời gian (s)	74,730	67,134	10,2%
Mẫu initial_alignment	209	93	55,5%
Thời gian alignment xấp xỉ (s)	10,45	4,65	55,5%
GT RMSE (cm)	3,185	3,346	-5,1%

Chu kỳ telemetry là 0,05 s nên 209 mẫu initial_alignment tương đương khoảng 10,45 s, còn 93 mẫu tương đương khoảng 4,65 s. Mức giảm 55,5% không đến từ việc tăng bừa $\omega_{\max}$; $\omega_{\max}$ vẫn là 0,70 rad/s. Nó đến từ việc giảm counter-rotation bằng bao phanh căn bậc hai theo $\alpha_{\text{eff}} = 0,18 \text{ rad/s}^2$.

Cần đọc kết quả narrow_aisles một cách thận trọng: GT RMSE tăng 5,1% và exit max tăng 0,6%. Vì vậy luật phanh mới chứng minh rõ việc giảm thời gian căn hướng và counter-rotation, nhưng chưa chứng minh mọi metric bám đường trên narrow_aisles đều tốt hơn. Các cột tăng được tô đỏ trong hình dưới.

Bảy trace cuối đều thành công, đạt ground-truth goal, dừng vật lý và giữ nominal jerk không quá 0,9 m/s³. Ground-truth RMSE nằm từ 1,570 đến 3,453 cm. Map open_arena có RMSE thấp nhất; warehouse_dispatch cao nhất, phù hợp với mật độ vật cản và số pha chuyển động phức tạp hơn.

Rà soát 26/07/2026: 100% là giá trị trước; xanh là giảm, đỏ là tăng



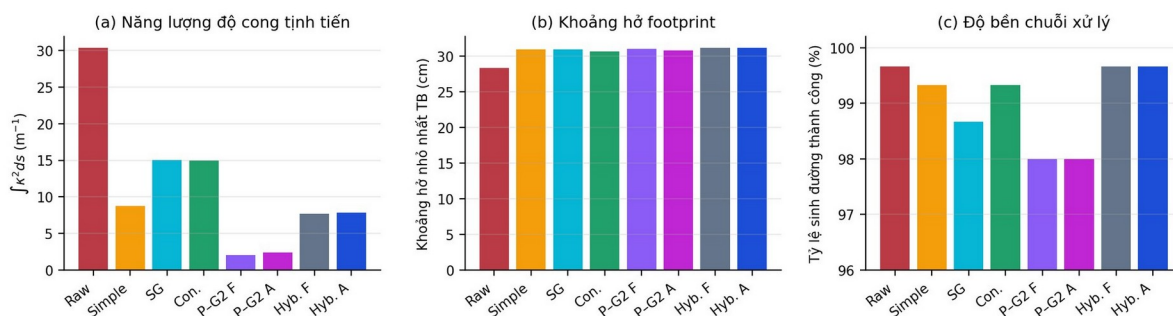
So sánh chuẩn hóa ba cặp before–after ngày 26/07/2026. Mốc 100% là trace trước; cột xanh là giảm, cột đỏ là tăng. Hình được sinh từ cùng JSON dùng để điền hai bảng phía trên.

12.4 So sánh hình học trên 7.200 dòng, ma trận ngày 25/07/2026

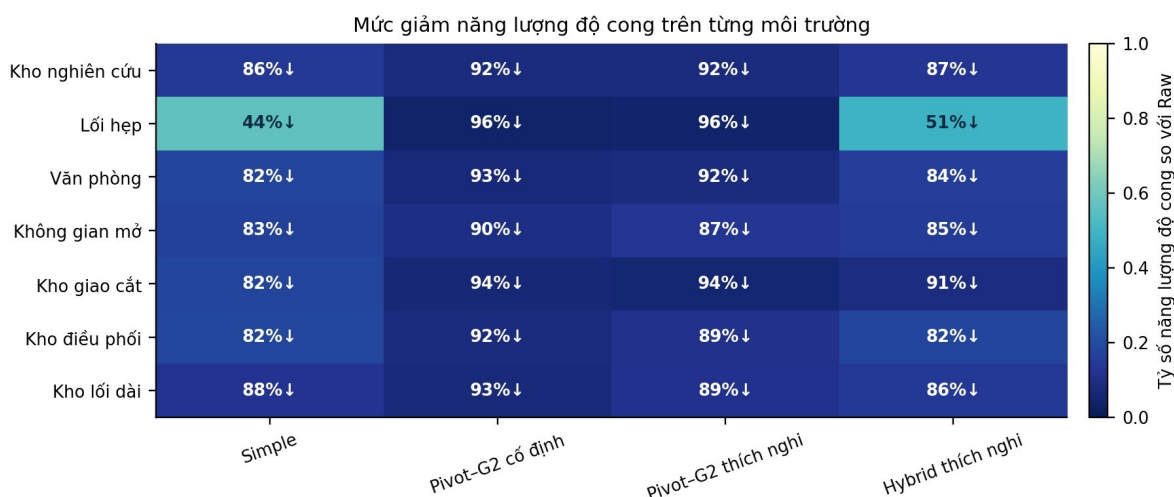
Phương pháp	Thành công	E_k (1/m)	Clearance (cm)	Chiều dài (m)	Lệch raw (cm)	Runtime (ms)
Raw	897/900	30,394	28,29	8,217	0,00	4,00
Nav2 Simple	894/900	8,790	30,94	8,162	0,56	0,69
Savitzky–Golay	888/900	15,067	30,94	8,158	0,11	0,14
Nav2 Constrained	894/900	15,004	30,64	8,218	0,56	10,87
Pivot–G ² cố định	882/900	2,074	31,01	8,087	1,76	2,94
Pivot–G ² thích nghi	882/900	2,378	30,80	8,082	1,86	6,09
Hybrid Pivot cố định	897/900	7,708	31,14	8,171	0,70	3,93
Adaptive Hybrid Pivot–G ²	897/900	7,825	31,15	8,170	0,69	7,11

Pure Pivot–G² thích nghi giảm 92,2% Ek so với Raw. Adaptive Hybrid giảm 74,3% nhưng đạt 897/900 và clearance trung bình 31,15 cm. Pure Pivot có Ek thấp hơn vì luôn ưu tiên nhánh tối ưu độ cong; Hybrid cố ý giữ Simple ở những ca Pivot không tạo đủ safety gain. Vì vậy không được dùng riêng Ek để kết luận phương pháp hoàn chỉnh kém hơn pure branch. Các giá trị Ek, clearance, chiều dài, độ lệch và runtime trong bảng là trung bình trên những hàng sinh đường thành công; số thành công trên 900 được trình bày riêng để phương pháp có nhiều ca fail không được che bởi một giá trị trung bình đẹp.

7.200 phép đo hình học: 7 map × 5 planner × 8 phương pháp × 3 lần



So sánh hình học từ 7.200 hàng của ma trận ngày 25/07/2026: năng lượng độ cong, clearance và tỷ lệ sinh đường thành công.



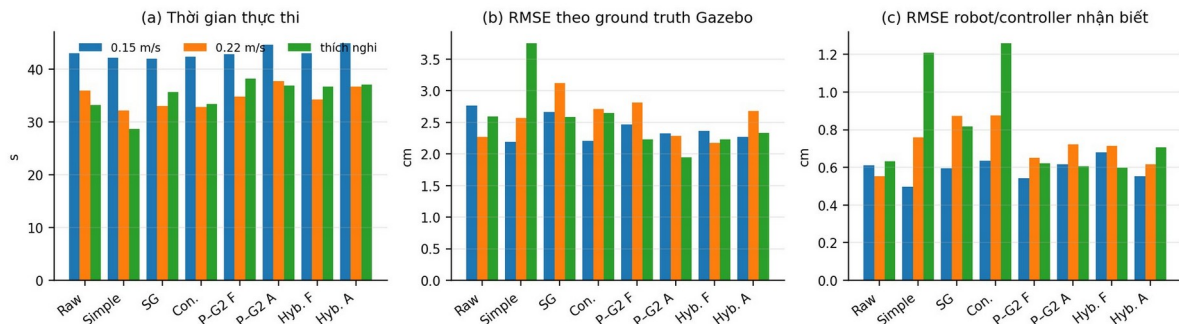
Mức giảm năng lượng độ cong theo từng môi trường và từng phương pháp, tính từ ma trận hình học ngày 25/07/2026.

12.5 Tám phương pháp trên lower_left_diagonal/ThetaStar, ma trận ngày 25/07/2026

Phương pháp	OK	t (s)	GT RMSE (cm)	GT max (cm)	Exit RMSE (cm)	v thực max (m/s)	Jerk nominal P95 (m/s ³)
Raw	Có	33,17	2,59	4,38	2,96	0,300	0,900
Nav2 Simple	Có	28,67	3,75	7,54	5,31	0,300	0,900
Savitzky–Golay	Có	35,67	2,59	4,62	3,31	0,300	0,900

Phương pháp	OK	t (s)	GT RMSE (cm)	GT max (cm)	Exit RMSE (cm)	v thực max (m/s)	Jerk nominal P95 (m/s ³)
Nav2 Constrained	Có	33,41	2,65	4,74	2,32	0,300	0,900
Pivot-G ² cố định	Có	38,18	2,23	3,77	2,86	0,300	0,900
Pivot-G ² thích nghi	Có	36,82	1,94	3,05	2,27	0,300	0,900
Hybrid Pivot cố định	Có	36,67	2,23	3,19	2,77	0,300	0,900
Adaptive Hybrid Pivot-G ²	Có	37,07	2,33	3,88	2,44	0,300	0,900

Chạy kín Gazebo: cùng planner ThetaStar và cùng bài lower_left_diagonal



Ảnh hưởng của smoother và chế độ tốc độ trong ma trận chạy kín ngày 25/07/2026. Bảng ngay phía trên chỉ lấy tám method có tốc độ thích nghi trên cùng research_warehouse/lower_left_diagonal/ThetaStar.

Trong scenario chính, Pivot-G² thích nghi có GT RMSE và exit RMSE tốt nhất trong nhóm nổi bật, nhưng kết luận không được dựa trên một scenario. Hệ thống cần được nhìn trên toàn ma trận, đặc biệt ở các ca hẹp và ca phản ví dụ.

12.6 Năm planner trên lower_left_diagonal, ma trận ngày 25/07/2026

Planner	OK	t (s)	GT RMSE (cm)	RMSE controller (cm)	Exit RMSE (cm)
NavFn A*	Có	48,42	2,41	0,62	2,60
NavFn Dijkstra	Có	38,88	2,47	0,62	2,96
Smac 2D	Có	30,65	3,19	1,38	3,53
Smac Hybrid	Có	38,12	4,31	1,45	5,49
Theta*	Có	36,92	2,42	0,71	2,86

Smoother không thay planner. Raw path khác nhau theo planner nên path cuối và hành vi controller cũng khác. Smac Hybrid không tự động cho kết quả tốt nhất trong bài toán robot vi

sai này, vì nó được thiết kế cho feasibility SE(2) rộng hơn và path của nó có thể tương tác khác với RPP và footprint. Kết luận đúng là phương pháp có thể làm việc với cả năm planner, không phải mọi planner cho cùng chất lượng.

12.7 Ca phản ví dụ phải được giữ lại trong báo cáo

Tại warehouse_dispatch/full_replenishment, ThetaStar kết hợp pure Pivot- G^2 ở 0,22 m/s thất bại vì RPP liên tục dự báo collision và kết thúc PATIENCE_EXCEEDED. Trên đúng cùng raw_path_sha256, Adaptive Hybrid chọn Simple, tăng clearance kế hoạch từ 10,61 cm lên 14,49 cm và hoàn thành trong 95,35 s.

Ca này chứng minh feasibility hình học tĩnh và executability vòng kín không phải một khái niệm. Một path có swept footprint clear vẫn có thể làm controller dự báo collision do cách lookahead, time-to-collision và sai số thực tế. Vì vậy tên phương pháp hoàn chỉnh phải nhấn mạnh Hybrid safety gate. Không nên tuyên bố pure Pivot luôn tốt nhất.

XIII, Đây là đóng góp riêng của dự án?

Điểm riêng mạnh nhất của dự án không phải việc sử dụng Bézier bậc năm, vì Bézier và tính liên tục G^2 đã tồn tại trong nghiên cứu trước. Đóng góp nằm ở cách toàn bộ hệ thống được ghép thành một chuỗi logic phù hợp với Nav2 và robot vi sai: neo lại start-goal liên tục để loại sai số tâm ô; xác định heading ban đầu theo occupancy map rồi xác nhận lại bằng selected path; tìm trim thích nghi ở từng góc; giữ Pivot như một trạng thái thực; dùng DP chống overlap; kiểm tra swept footprint; fallback bằng Hybrid gate; tạo speed envelope hai chiều; sửa projection theo hướng; phanh góc theo khả năng giảm tốc đo từ Gazebo; phân loại đúng zero-speed safety override; thực thi terminal servo; hiệu chuẩn odometry tách khỏi hình học CAD; và đánh giá bằng ground truth phân tầng.

Thành phần	Nguồn	Vai trò trong hệ thống
ROS 2/Nav2/Gazebo/RViz2	Nền tảng mã nguồn mở	Middleware, navigation server, physics và visualization.
NavFn/Theta*/Smac	Nav2	Sinh raw path.
Simple/SG/Constrained	Baseline	Đối chứng và nhánh Simple trong Hybrid.
Bézier bậc năm G^2	Cơ sở toán học có trước	Dạng transition được hiện thực và ràng buộc theo robot.
Adaptive trim search + DP + diagnostics	Phát triển của dự án	Tự chọn candidate toàn đường.
Safety-gated Hybrid + Raw fallback	Phát triển của dự án	Tăng độ robust và không xuất đường nguy hiểm.
Bidirectional jerk-aware speed envelope	Phát triển của dự án	Phanh trước và tăng lại sau cong khả thi.
Heading-aware projection + terminal servo	Phát triển của dự án	Sửa nhảy nhánh, lệch sau cong và sai số goal.
Ground-truth benchmark phân tầng	Phát triển của dự án	Tách lỗi vật lý, định vị và controller.

XIV, Giới hạn hiện tại và hướng phát triển tiếp

Thuật toán chưa có bằng chứng tối ưu toàn cục trong không gian liên tục. Coarse-to-fine có evaluation budget nên chỉ tìm nghiệm tốt trong miền đã lấy mẫu. DP tối ưu chuỗi candidate đã sinh, không tối ưu đồng thời mọi biến liên tục của toàn đường.

Mô phỏng không thay thế robot thật. Ma sát sàn, backlash hộp số GA25, tải hàng, sụt áp của pack 4S4P, độ trễ driver, nhiễu lidar, sai số IMU và biến dạng bánh xe ngoài thực tế khác Gazebo. Trước khi chạy thật cần đo lại footprint, L, bán kính bánh, vận tốc bánh tối đa và hệ số acceleration. Đặc biệt, PPR, vị trí encoder và chế độ quadrature hiện chưa biết nên chưa được phép tính metres_per_tick. Model BMS, fuse, buck 12 V và motor driver cũng phải được chốt bằng datasheet trước khi cấp nguồn; giá trị 52 A của cell-array không phải rating mặc định của xe.

Ma trận closed-loop (vòng kín có phản hồi) hiện là thiết kế phân tầng, chưa phải toàn bộ tích 7 map $\times$ 5 planner $\times$ 8 smoother $\times$ 3 tốc độ $\times$ nhiều seed. Các adaptive parameter đã được tuning trên cùng họ map nên cần có hold-out map và nhiều seed để đánh giá khả năng tổng quát.

Footprint hiện dùng hình chữ nhật bảo thủ. Đây là lựa chọn an toàn nhưng có thể loại nhầm các đường hợp lệ quanh hốc bánh hoặc cạnh vát. Một hướng phát triển là dùng polygon sát CAD hơn, nhưng phải kiểm tra chi phí tính toán và độ ổn định của collision checker.

RPP collision prediction có thể hủy một path hình học vẫn clear. Hướng phát triển tiếp theo là đưa executability model vào gần hơn với objective hình học, ví dụ dự báo time-to-collision hoặc khả năng bám của controller ngay trong bước xếp hạng candidate.

Sau khi ổn định trên robot thật, dự án có thể mở rộng sang event-triggered replanning. Khi costmap thay đổi, thay vì lập lại toàn bộ ở mọi chu kỳ, hệ thống chỉ yêu cầu replan khi đường hiện tại mất an toàn hoặc chi phí tăng đủ lớn. Một hướng khác là đưa năng lượng điện thực, dòng motor và tải hàng vào hàm cost thay cho E_k xấp xỉ.

XV, Cách build, chạy, quan sát và debug

15.1 Build workspace

```
cd /home/linh-pham/agv_nav2_research_ws
source /opt/ros/jazzy/setup.bash
colcon build --symlink-install
source install/setup.bash
ros2 launch vacuum_robot_gazebo switchable_simulation.launch.py gui:=true
```

15.2 Quy trình chạy cho người mới

Sau khi stack active, trong panel RViz2 chọn môi trường và nhấn đổi map/world. Đợi lifecycle manager đưa Nav2 về trạng thái active. Chọn planner cần thử. Dùng 2D Goal Pose để click-drag vị trí và yaw goal. Bật hiển thị tất cả smoother để xem cùng một raw path được biến đổi như thế nào, sau đó ẩn từng đường để quan sát riêng. Khi muốn chạy robot, chọn execute method là adaptive_hybrid và theo dõi path được chọn, ground truth, /cmd_vel cùng telemetry.

Khi một trial thất bại, không nên chỉ nhìn message cuối. Cần kiểm tra theo thứ tự: planner có trả path hay không, smoother chọn nhánh nào, candidate bị loại bởi hard constraint nào, footprint clearance bao nhiêu, speed cap bị giới hạn bởi nguồn nào, projection đang ở s nào, RPP có báo collision hay không và terminal servo dừng ở pha nào.

15.3 Các topic hữu ích

Topic	Nội dung
/research/goal_pose	Goal nghiên cứu từ RViz2
/planner_selector	Planner đang chọn
/research/smoothier_visibility	Các đường đang bật hoặc ẩn
/research/environment_selector	Yêu cầu đổi world/map
/research/metrics	Metric của từng đường
/research/adaptive_speed	Cap, phase, sai số và safety override
/cmd_vel	Lệnh vận tốc cuối
/odom	Wheel odometry
/ground_truth/odom	Pose vật lý Gazebo
/scan	Dữ liệu lidar

15.4 Test package

```
colcon test --packages-select adaptive_pivot_g2 adaptive_pivot_g2_nav2  
adaptive_pivot_g2_controller adaptive_pivot_g2_benchmark adaptive_pivot_g2_rviz  
vacuum_robot_gazebo  
colcon test-result --all --verbose
```

XVI, Kết luận

Dự án bắt đầu từ một câu hỏi tương đối đơn giản: tại sao global planner đã tìm được đường mà robot vẫn đi không tốt? Khi phân tích sâu hơn, câu trả lời nằm ở khoảng trống giữa path hình học và chuyển động thực. Global planner không chịu trách nhiệm toàn bộ cho cách robot vào cua, phanh, quay và tăng tốc lại. Nếu để controller tự giải quyết mọi góc gấp, hành vi phụ thuộc quá nhiều vào tham số và dễ mất ổn định khi thay môi trường.

Adaptive Hybrid Pivot- G^2 giải quyết khoảng trống đó theo một chuỗi kín. Raw path được neo đúng start-goal liên tục rồi điều kiện hóa. Hướng ban đầu được suy ra từ map và được controller xác nhận lại theo preview của selected path. Mỗi góc có trạng thái Pivot và nhiều ứng viên G^2 . Tìm kiếm thích nghi chọn trim, DP giải xung đột toàn đường, swept footprint loại va chạm, Hybrid gate giữ Simple hoặc Raw khi cần. Sau đó speed envelope hai chiều, luật phanh góc và maneuver-aware RPP biến path thành lệnh vận tốc có thể thực hiện. Benchmark tách hình học, controller, odometry, định vị và ground truth để tránh kết luận dựa trên hình ảnh RViz2.

Pure Pivot- G^2 là lỗi tối ưu độ cong. Adaptive Hybrid Pivot- G^2 mới là phương pháp hoàn chỉnh để chạy robot, bởi nó chấp nhận rằng không có một nhánh nào tốt nhất trong mọi bản đồ. Kết quả 7.200 phép đo hình học, 41/42 trial vòng kín, bảy môi trường và năm planner cho thấy hướng nghiên cứu có giá trị, đồng thời ca phản ví dụ cũng chỉ ra rõ phần còn phải tiếp tục cải thiện trước khi triển khai quy mô lớn trên robot thật.

Kết luận ngắn nhất của toàn dự án: đường Bézier đẹp chỉ là một phần. Đóng góp thực sự là chuỗi quyết định từ candidate G^2 /Pivot đến fallback, profile vận tốc, controller và benchmark ground truth.

XVII, Nguồn lý thuyết và cách sử dụng hình minh họa

Phần lý thuyết nền được đối chiếu với tài liệu chính thức của ROS 2 và Navigation2. Cấu trúc nav_msgs/Path, PoseStamped, Planner Server, Smoother Server và Controller Server được đối chiếu trực tiếp từ tài liệu ROS 2 Jazzy và Nav2. Phần Regulated Pure Pursuit (Pure Pursuit có điều tiết vận tốc) được đối chiếu với bài báo của Macenski và cộng sự cùng tài liệu cấu hình chính thức của Nav2. Phần Bézier bậc năm và quỹ đạo trơn cho robot nonholonomic được đối chiếu với công trình của Simba, Uchiyama và Sano.

Các hình nền tảng như path-trajectory, $G^0/G^1/G^2$, các bậc Bézier, hình học trim, coarse-to-fine (tìm thô trước rồi tinh chỉnh), DP, Hybrid gate, speed envelope và Pure Pursuit trong bản này không sao chép nguyên hình trên mạng. Chúng được tự vẽ lại bằng đường nét đen trắng dựa trên định nghĩa toán học và cấu trúc thuật toán để bảo đảm thống nhất ký hiệu, dễ in và không phụ thuộc chất lượng ảnh của nguồn bên ngoài. Những hình vẽ robot CAD, giao diện RViz2/Gazebo, bảy bản đồ và biểu đồ benchmark được lấy từ báo cáo toàn diện của chính dự án rồi chuyển sang thang xám.

Nguồn đối chiếu	Phần được sử dụng
ROS 2 Jazzy nav_msgs/Path và geometry_msgs/PoseStamped	Định nghĩa message chứa chuỗi pose mà planner trả về.
Nav2 Navigation Concepts và Navigation Plugins	Kiến trúc plugin, vai trò planner, smoother, controller và behavior.
Nav2 Planner Server / Smoother Server / Controller Server	Action server, costmap, footprint và cách nạp plugin.
K. R. Simba, N. Uchiyama, S. Sano, RCIM 41 (2016)	Cơ sở dùng Bézier để tạo chuyển động trơn cho robot nonholonomic.
S. Macenski et al., Autonomous Robots 47 (2023)	Cơ sở của Regulated Pure Pursuit và điều tiết vận tốc theo an toàn.
H. Pham, Q.-C. Pham, time-optimal path parameterization	Cơ sở phân biệt path hình học và time-parameterization dưới ràng buộc động học.

PHỤ LỤC A, Thuật ngữ dùng xuyên suốt dự án

Thuật ngữ	Giải nghĩa trong dự án
Adaptive	Tham số được chọn theo từng dữ liệu đầu vào thay vì một giá trị cố định.
AMCL	Định vị robot trên map bằng particle filter.
Baseline	Phương pháp đối chứng dùng làm mốc so sánh.
Clearance	Khoảng hở nhỏ nhất từ toàn footprint đến vật cản.
Closed loop	Lệnh được cập nhật bằng phản hồi trạng thái robot.
Coarse-to-fine	Tìm kiếm thô trước rồi tinh chỉnh vùng đáng chú ý.
Costmap	Lưới chi phí 2D của Nav2.
Curvature κ	Mức thay đổi hướng tiếp tuyến trên một mét đường.
Dynamic Programming	Ghép nghiệm tối ưu từ trạng thái con và quan hệ tương thích.
Fallback	Phương án dự phòng khi nhánh ưu tiên không hợp lệ.
Footprint	Đa giác chiếu bằng đại diện toàn thân robot.
G ² continuity	Liên tục vị trí, hướng tiếp tuyến và độ cong.
Ground truth	Pose vật lý lấy trực tiếp từ Gazebo, không qua AMCL/odom.
Hard constraint	Vi phạm là loại ứng viên, không được cost khác bù.
Hybrid	Chọn giữa nhiều nhánh theo luật đã công bố.
Jerk	Đạo hàm của gia tốc.
Marker Pivot	Hai pose cùng vị trí nhưng khác yaw, biểu diễn dừng và quay tại chỗ.
Path	Chuỗi pose hình học, chưa nhất thiết có thời gian.
Projection	Tìm tiến độ tương ứng của robot trên path.
Raw path	Đường planner trả về trước mọi bước làm mượt.
RPP	Pure Pursuit có thêm điều tiết vận tốc theo cong, cost và va chạm dự báo.
Safety gate	Cổng chỉ cho một nhánh đi qua khi thỏa điều kiện.
Smoother	Bộ hậu xử lý path của planner.
Swept footprint	Hợp của footprint khi robot quét dọc chuyển động.
Time parameterization	Gắn vận tốc và thời gian vào path dưới các giới hạn động học.
Trajectory	Path đã có quy luật thời gian, vận tốc và thường cả gia tốc.
4S4P	Bốn cell nối tiếp trong mỗi nhánh và bốn nhánh song song; tổng 16 cell.
Buck converter	Bộ hạ áp DC–DC; trong xe dùng để tạo rail 12 V từ pack 4S tối đa 16,8 V.
C-rate	Tỷ lệ dòng xả theo dung lượng Ah; 5C của cell 2,6 Ah tương ứng 13 A.

Thuật ngữ	Giải nghĩa trong dự án
CW/CCW	Clockwise/counter-clockwise, quay thuận hoặc ngược chiều kim đồng hồ.
Encoder PPR	Số xung trên một vòng theo định nghĩa phần cứng; hiện chưa biết và không được suy đoán.
Effective wheel separation	Khoảng vệt lăn hiệu dụng dùng để hiệu chuẩn contact/odometry Gazebo, không thay kích thước CAD.
Initial alignment	Pha dừng và quay để hướng thân xe khớp tiếp tuyến đầu selected path trước khi chạy tiến.
Locked rotor / stall	Trạng thái trục motor bị khóa; dòng và nhiệt tăng cao, không phải chế độ vận hành.
No-load speed	Tốc độ khi motor gần như không mang tải; không phải tốc độ chạy liên tục của xe.
Rated-load	Điểm làm việc có tải danh nghĩa dùng để đánh giá vận hành liên tục.
Safety override	Can thiệp bắt buộc khi cap an toàn hoặc biên $v = 0$ phải thẳng giới hạn jerk danh nghĩa.
Start/goal anchoring	Phục hồi hai đầu path về đúng pose liên tục trong một ngưỡng kiểm tra.

PHỤ LỤC B, Vị trí source chính và tài liệu tham khảo

Nội dung	File chính
Bézier G^2	src/adaptive_pivot_g2/src/quintic_transition.cpp
Adaptive search	src/adaptive_pivot_g2/src/adaptive_search.cpp
Quy hoạch động	src/adaptive_pivot_g2/src/path_optimization.cpp
Hybrid gate	src/adaptive_pivot_g2/src/hybrid_selection.cpp
Nav2 smoother	src/adaptive_pivot_g2_nav2/src/
Speed envelope	src/adaptive_pivot_g2_controller/src/adaptive_speed_profile.cpp
Maneuver-aware RPP	src/adaptive_pivot_g2_controller/src/ maneuver_aware_rpp_controller.cpp
Benchmark	src/adaptive_pivot_g2_benchmark/
Cấu hình Nav2	src/vacuum_robot_gazebo/config/nav2_params.yaml
Map/world	src/vacuum_robot_gazebo/maps/ và worlds/
Neo start/goal liên tục	src/adaptive_pivot_g2_benchmark/ adaptive_pivot_g2_benchmark/path_contract.py
Định hướng ban đầu map-aware	src/adaptive_pivot_g2_benchmark/ adaptive_pivot_g2_benchmark/initial_heading.py
Phanh góc và preview heading	src/adaptive_pivot_g2_controller/src/maneuver_path.cpp
State machine initial alignment	src/adaptive_pivot_g2_controller/src/ maneuver_aware_rpp_controller.cpp
Zero-speed override	src/adaptive_pivot_g2_controller/src/ adaptive_speed_profile.cpp
Hợp đồng GA25 và nguồn 4S4P	src/vacuum_robot_gazebo/config/real_robot_profile.yaml
Mô hình URDF	src/vacuum_robot_gazebo/urdf/vacuum_robot.urdf
Mô hình SDF và DiffDrive	src/vacuum_robot_gazebo/models/vacuum_robot/ model.sdf
Dữ liệu kiểm chứng hiện tại	results/current_full_audit_20260726/*.json.gz
Manifest hình theo dữ liệu hiện tại	docs/bao_cao_toan_dien_assets/ current_figure_manifest.json
Ma trận hình học lịch sử	results/conference_geometry_20260725/*.csv
Ma trận chạy kín lịch sử	results/conference_execution_20260725/ conference_execution_compact.csv
Bảng legacy before–after	docs/ REV_ECIT_2026_ADAPTIVE_HYBRID_PIVOT_G2_SUPPL EMENT.html
Dựng lại báo cáo chi tiết	tools/rebuild_detailed_algorithm_report.py
Xuất PDF và HTML tự chứa hình	tools/export_detailed_algorithm_report.py

Tài liệu tham khảo chính của dự án và phần lý thuyết nền gồm: K. R. Simba, N. Uchiyama và S. Sano, “Real-time smooth trajectory generation for nonholonomic mobile robots using Bézier curves”, Robotics and Computer-Integrated Manufacturing, 2016; S. Macenski và cộng sự, “Regulated pure pursuit for robot path tracking”, Autonomous Robots, 2023; H. Pham và Q.-C. Pham, “A new approach to time-optimal path parameterization based on reachability

analysis”; cùng tài liệu chính thức của Navigation2 về planner, smoother, controller, costmap, AMCL và Behavior Tree.

Nguồn dữ liệu, hình ảnh và các kết quả định lượng trong báo cáo này được tổng hợp từ ma trận nghiên cứu ngày 25/07/2026, đợt kiểm chứng `current_full_audit_20260726` và mã nguồn hiện tại của workspace `agv_nav2_research_ws`. Các bảng lịch sử được ghi rõ phiên bản; bảng bày map hiện tại lấy trực tiếp từ các trace `*_pivot_g2_final.json.gz`.